Proyecto MLOps: Clasificador de Exoplanetas Kepler

Evidencias de las prácticas del curso de MLOps

Autora: María Fernanda González Rodríguez (Mafer) · Código 220573562

Carrera: Ingeniería en Informática · CUCEI, Universidad de Guadalajara

Repositorio: https://github.com/Mafersita-code/exoplanet-cicd

Dataset: Kepler Objects of Interest (*cumulative.csv*, NASA Exoplanet Archive). El modelo, un Random Forest, clasifica cada objeto como CONFIRMED o FALSE POSITIVE.

Contenido

Parte	Contenido	Página
Práctica 8 - CI/CD	FastAPI, pytest, Docker, GitHub Actions y GitHub Container Registry	2
Práctica 9 - Monitoreo de modelos	Detección de drift con Evidently AI	11
Kubernetes - Autoescalado	Docker Desktop Kubernetes, metrics-server y HPA con prueba de carga	20
AWS - Despliegue en la nube	S3, DVC, ECR, ECS/Fargate, CloudWatch, GitHub Actions, MLflow y Airflow en EC2	28

Recorrido del proyecto: la API del modelo se construyó con FastAPI, se probó con pytest y se empaquetó en Docker con CI/CD en GitHub Actions (Práctica 8). Después se monitoreó el drift de los datos con Evidently (Práctica 9), se demostró el autoescalado con Kubernetes y, finalmente, todo el flujo se llevó a AWS: datos y modelos en S3 con DVC, imagen en ECR, API en ECS/Fargate con CloudWatch, y MLflow y Airflow corriendo en un servidor EC2.

Práctica 8 - CI/CD: Clasificador de Exoplanetas

FastAPI + pytest + Docker + GitHub Actions + GitHub Container Registry

Autora: Mafer González

Repositorio: https://github.com/Mafersita-code/exoplanet-cicd

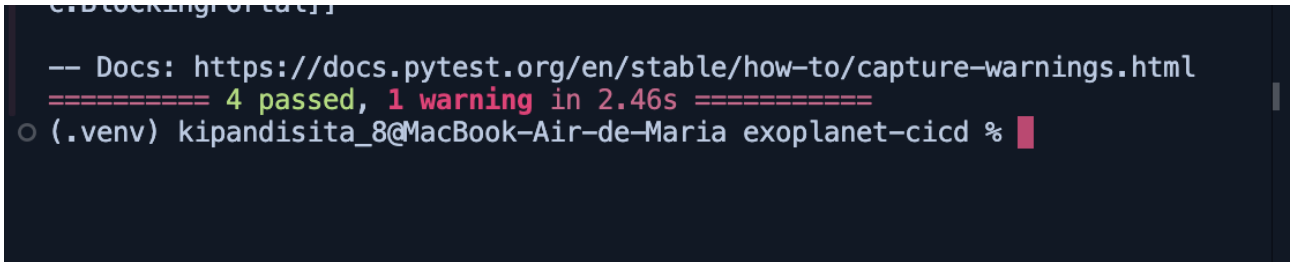
Imagen publicada: `ghcr.io/mafersita-code/exoplanet-cicd`

Dataset: Kepler Objects of Interest (cumulative.csv, NASA Exoplanet Archive). El modelo clasifica cada objeto como CONFIRMED o FALSE POSITIVE.

N.º	Evidencia	Qué demuestra
1	Terminal con pytest local	Las 4 pruebas pasan al inicio.
2	FastAPI local en /docs	La API corre localmente con uvicorn (sin Docker) y responde a POST /predict.
3	Salida de docker images	La imagen local exoplanet-api:v1 existe.
4	Repositorio de GitHub	El proyecto y el workflow (.github/workflows) están subidos.
5	Actions exitoso inicial	El primer push a main ejecutó el workflow con éxito.
6	Actions fallido	El commit 'Introducir fallo de prueba' (200 cambiado a 404) hace fallar el job test.
7	Job docker omitido	Como test falló, docker queda omitido (skipped) por needs: test y no se publica imagen.
8	Actions exitoso corregido	Tras restaurar 200, test y docker pasan.
9	Paquete en GHCR	La imagen está publicada con las etiquetas latest y el SHA del commit.
10	API desde la imagen de GHCR	La imagen descargada de ghcr.io/mafersita-code/exoplanet-cicd arranca y predice.

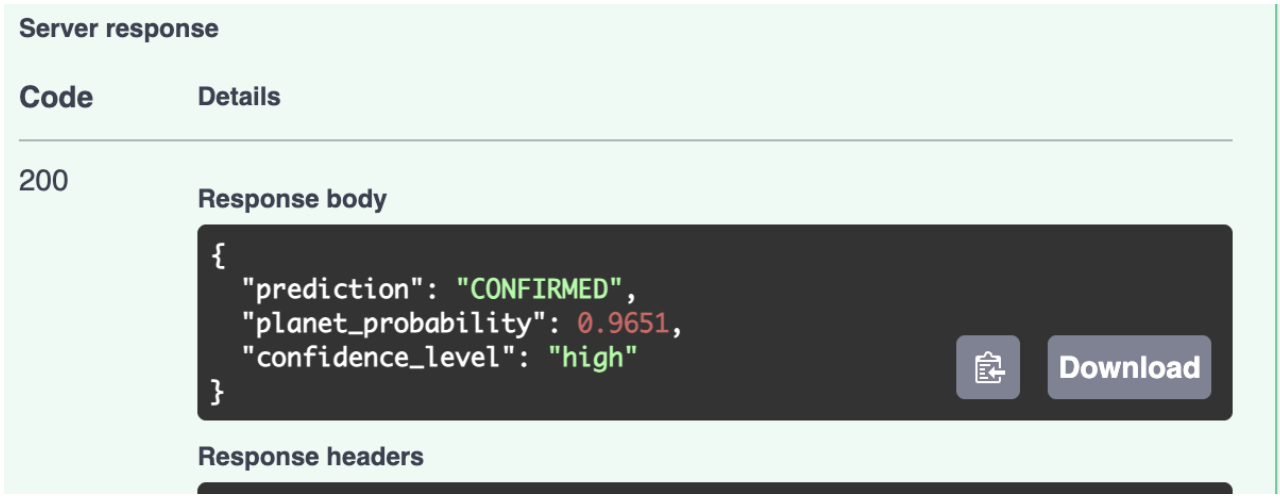
Evidencia 1. Terminal con pytest local

Las 4 pruebas pasan al inicio.



Evidencia 2. FastAPI local en /docs

La API corre localmente con uvicorn (sin Docker) y responde a POST /predict.



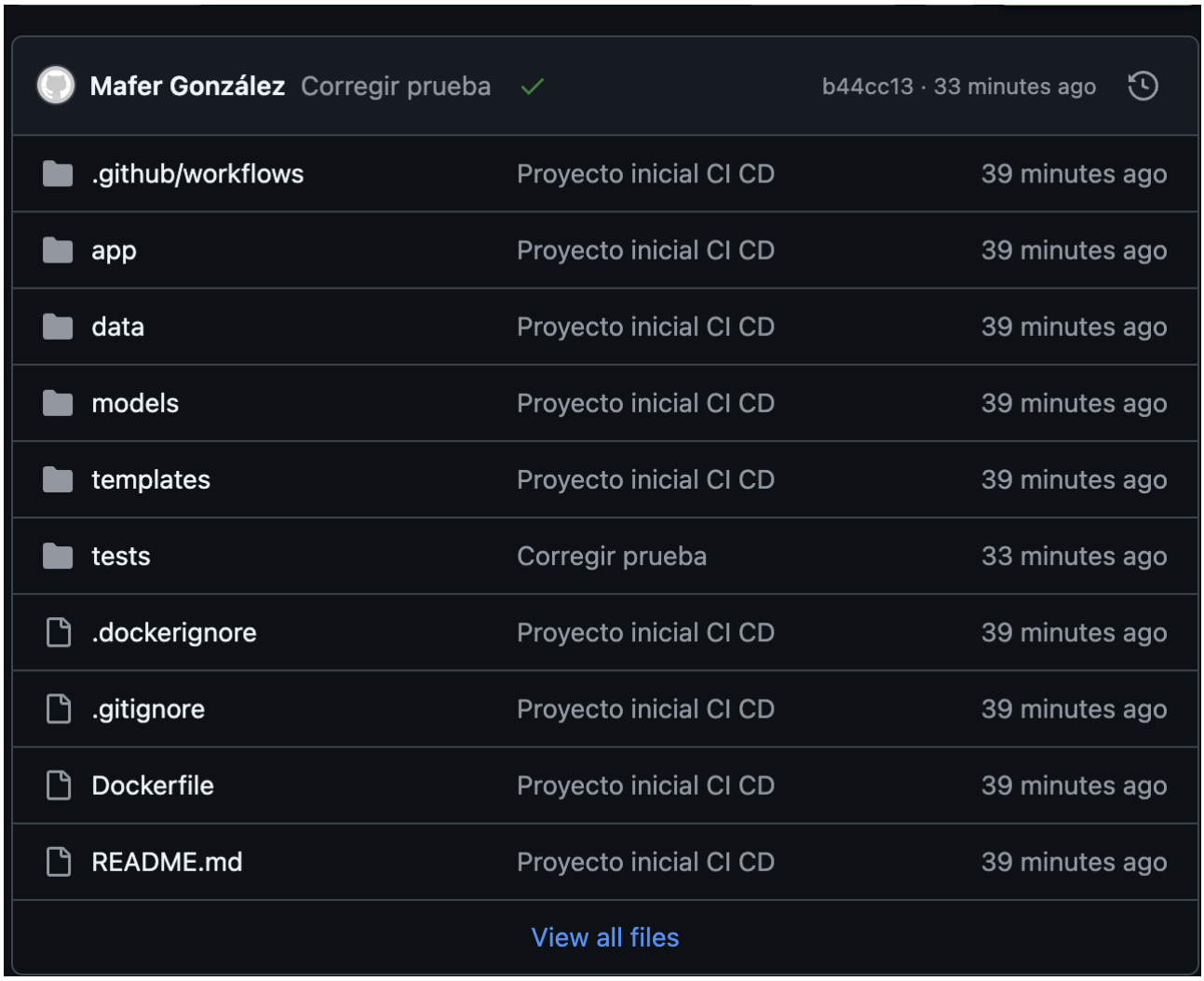
Evidencia 3. Salida de docker images

La imagen local exoplanet-api:v1 existe.



Evidencia 4. Repositorio de GitHub

El proyecto y el workflow (.github/workflows) están subidos.



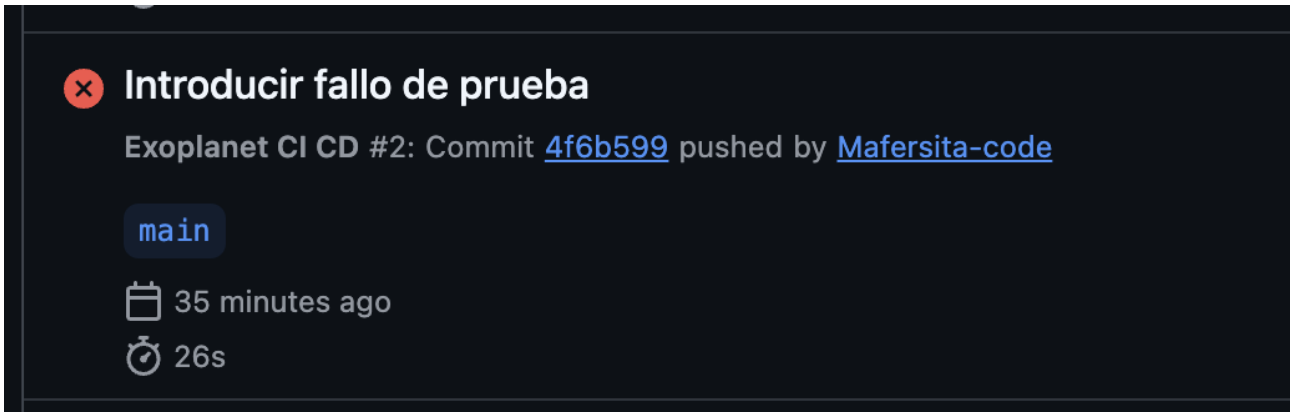
Evidencia 5. Actions exitoso inicial

El primer push a main ejecutó el workflow con éxito.



Evidencia 6. Actions fallido

El commit 'Introducir fallo de prueba' (200 cambiado a 404) hace fallar el job test.



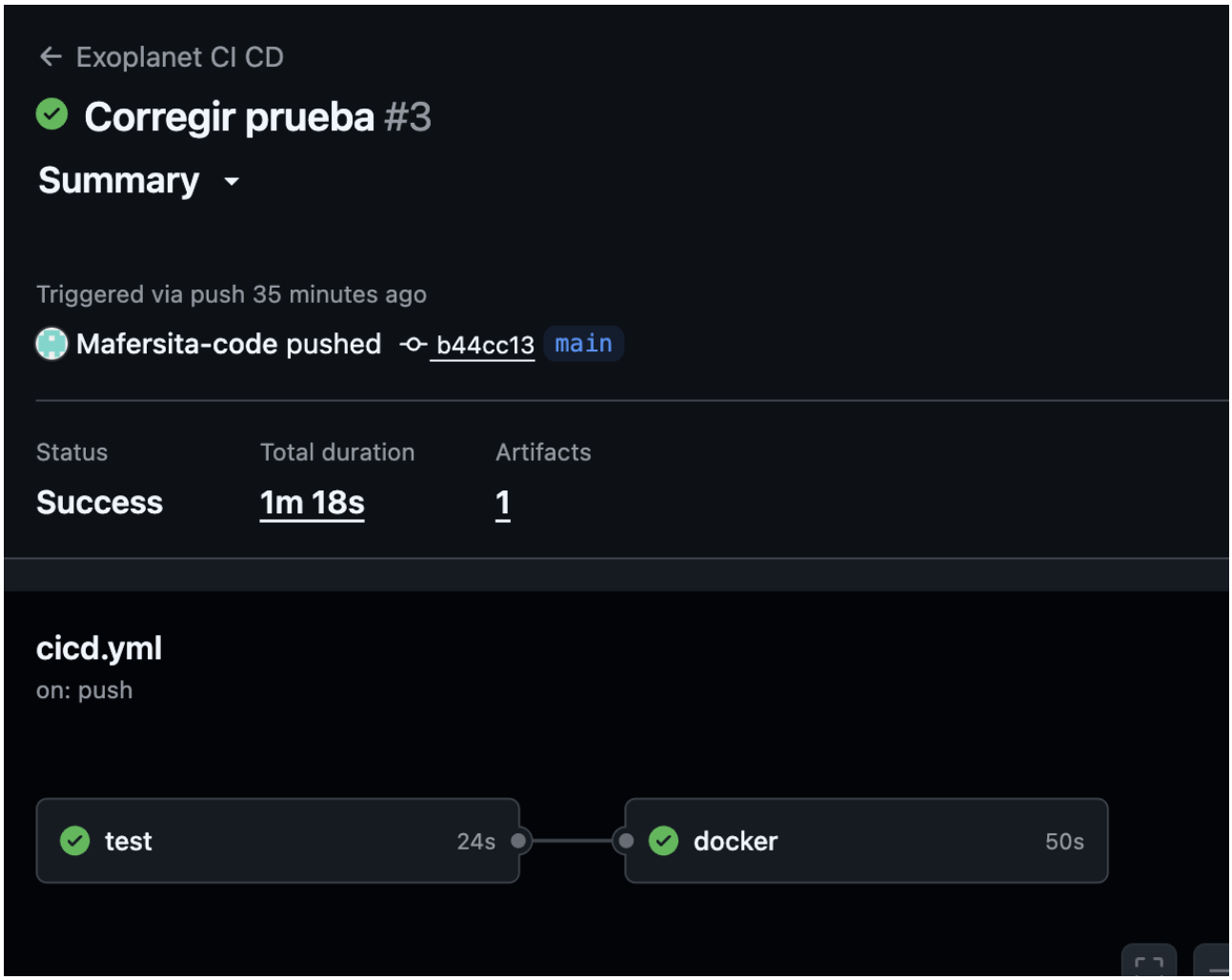
Evidencia 7. Job docker omitido

Como test falló, docker queda omitido (skipped) por needs: test y no se publica imagen.



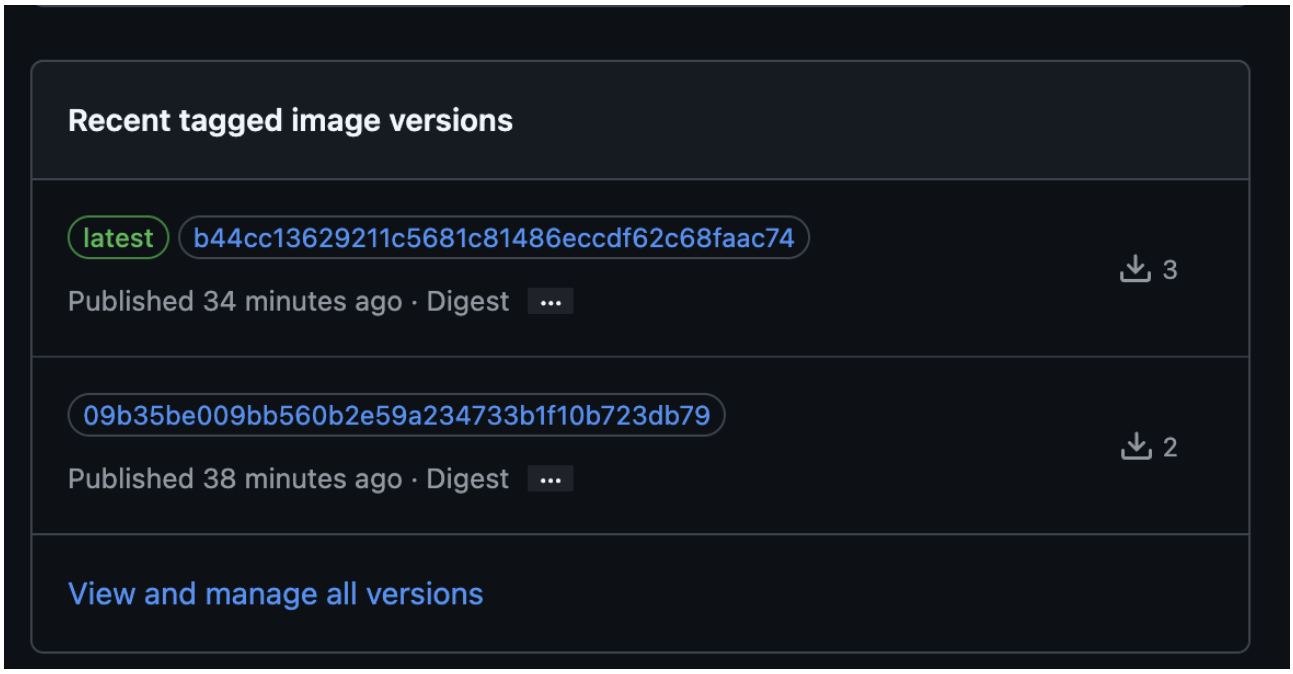
Evidencia 8. Actions exitoso corregido

Tras restaurar 200, test y docker pasan.



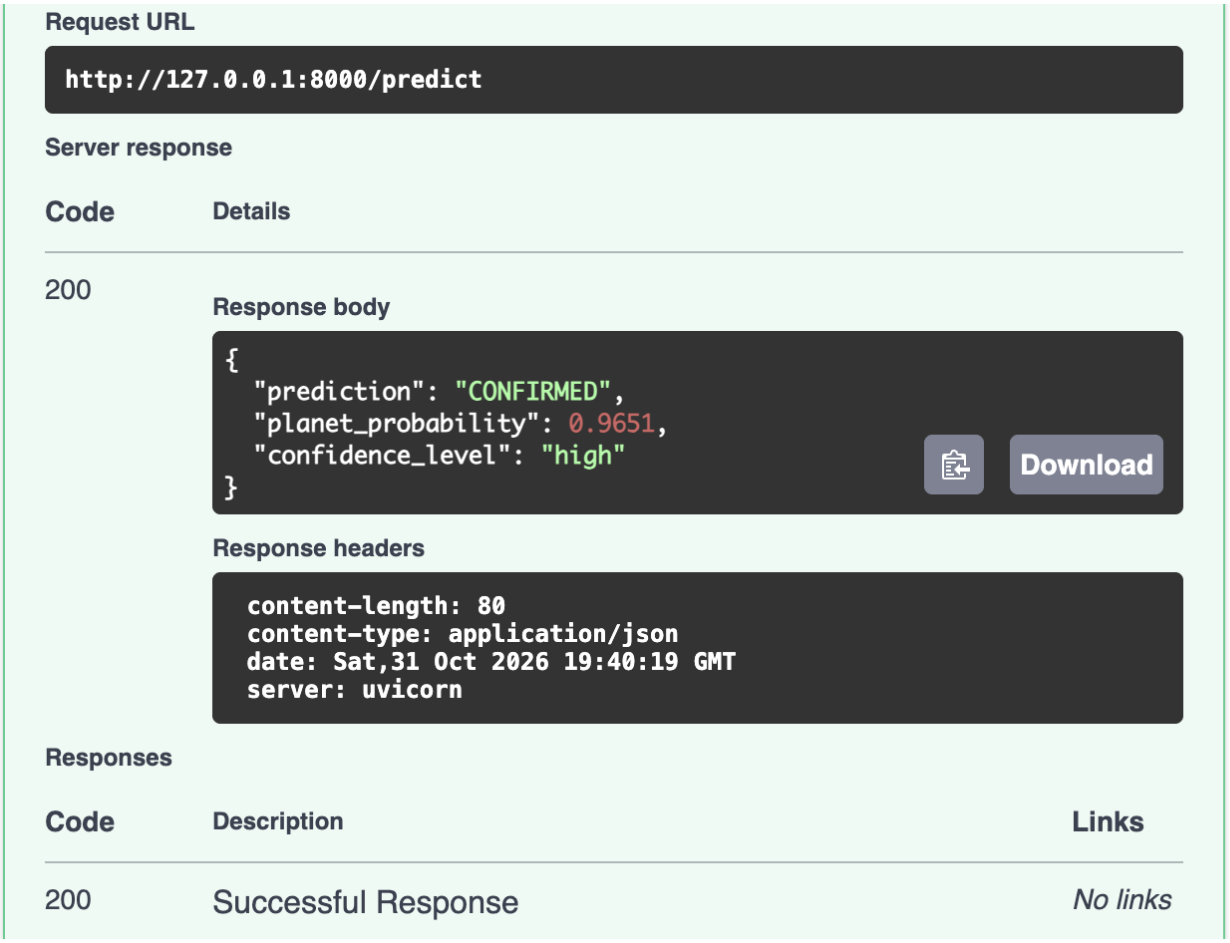
Evidencia 9. Paquete en GHCR

La imagen está publicada con las etiquetas latest y el SHA del commit.



Evidencia 10. API desde la imagen de GHCR

La imagen descargada de ghcr.io/mafersita-code/exoplanet-cicd arranca y predice.



Reflexión - Práctica 8: CI/CD con el clasificador de exoplanetas

Repositorio: https://github.com/Mafersita-code/exoplanet-cicd

Imagen publicada: `ghcr.io/mafersita-code/exoplanet-cicd` (etiquetas `latest` y SHA del commit)

1. ¿Qué problema evita CI?

La integración continua evita que un cambio defectuoso llegue al proyecto sin que nadie lo note. Cada vez que hago push a `main`, GitHub Actions instala las dependencias en una máquina limpia y ejecuta las pruebas con pytest. Así no dependo de "en mi computadora funciona": si olvido una dependencia en `requirements.txt`, rompo el endpoint `/predict` o cambio la validación de los datos (por ejemplo, aceptar un periodo orbital negativo), el error aparece en minutos y queda asociado al commit que lo causó.

2. ¿Por qué se bloquea la publicación si falla pytest?

Porque el job `docker` tiene `needs: test`: solo se ejecuta si las pruebas terminaron bien. En la demostración cambié la prueba de `/health` para que esperara 404 en lugar de 200; el job `test` falló y el job `docker` quedó *skipped*, así que no se publicó ninguna imagen nueva en GHCR. Esto protege a quien use la imagen: la última versión publicada siempre corresponde a un código que pasó las pruebas. Al corregir la prueba, ambos jobs volvieron a pasar y se publicó una imagen nueva.

3. ¿Qué diferencia hay entre la imagen y el contenedor?

La **imagen** es el paquete inmutable: contiene Python 3.11, las librerías, el código de la API y el modelo `champion.joblib`. Es como una plantilla. El **contenedor** es una instancia en ejecución de esa imagen: un proceso con su propio sistema de archivos y red, que puedo iniciar con `docker run` y detener. De una misma imagen (`exoplanet-api:v1` o la de GHCR) puedo crear muchos contenedores; al usar `--rm`, el contenedor se elimina al detenerlo, pero la imagen permanece.

4. ¿Por qué publicar una imagen no equivale a desplegarla automáticamente en producción?

Publicar la imagen en GHCR es **entrega continua** (*continuous delivery*): deja listo un artefacto probado y versionado que *se puede* desplegar. El **despliegue continuo** (*continuous deployment*) requeriría un paso adicional que tome esa imagen y la ejecute automáticamente en un servidor o servicio en la nube, con su configuración, red, escalado y monitoreo. En esta práctica ese paso no existe: yo descargué la imagen y la ejecuté manualmente en mi computadora para verificarla. Además, en un caso real se suele aprobar el despliegue o revisar métricas del modelo antes de exponerlo a usuarios.

Nota sobre el modelo

El modelo clasifica objetos de interés de Kepler como CONFIRMED o FALSE POSITIVE usando 8 variables físicas (periodo, duración y profundidad del tránsito, radio y temperatura del planeta, señal/ruido, temperatura y radio de la estrella). En el conjunto de prueba obtuvo Accuracy 0.913, F1 0.866 y ROC-AUC 0.968. Excluí `koi_score` y las banderas `koi_fpflag_*` porque prácticamente revelan la respuesta. Es un modelo didáctico, no apto para decisiones científicas reales.

Práctica 9 - Monitoreo de modelos con Evidently AI

Clasificador de Exoplanetas Kepler

Autora: Mafer González

Dataset: Kepler Objects of Interest (cumulative.csv, NASA Exoplanet Archive).

Diseño: el número de KOI refleja el orden de descubrimiento. El modelo Champion (Random Forest) se entrenó con el periodo temprano (KOI < 2000) y se monitorea sobre los objetos descubiertos después (KOI >= 2000).

Herramientas: Evidently AI 0.7, pandas, scikit-learn.

N.º	Evidencia
1	Datasets de referencia y actuales
2	Reporte HTML de Evidently
3	Variables con drift detectado
4	Comparación de distribuciones
5	Comparación de predicciones
6	Comparación de Accuracy, F1 y ROC-AUC
7	Alerta por deterioro
8	Interpretación de los resultados y decisión recomendada

Evidencia 1. Datasets de referencia y actuales

Referencia: 690 objetos del periodo temprano (KOI < 2000) con etiqueta. Actual: 6,820 objetos del periodo posterior (KOI ≥ 2000), sin etiqueta.

```
data/current.csv'); print(a.shape, b.shape)"
(690, 9) (6820, 8)
(.venv) kipandisita_8@MacBook-Air-de-Maria exoplanet-monitoring %
```

Evidencia 2. Reporte HTML de Evidently

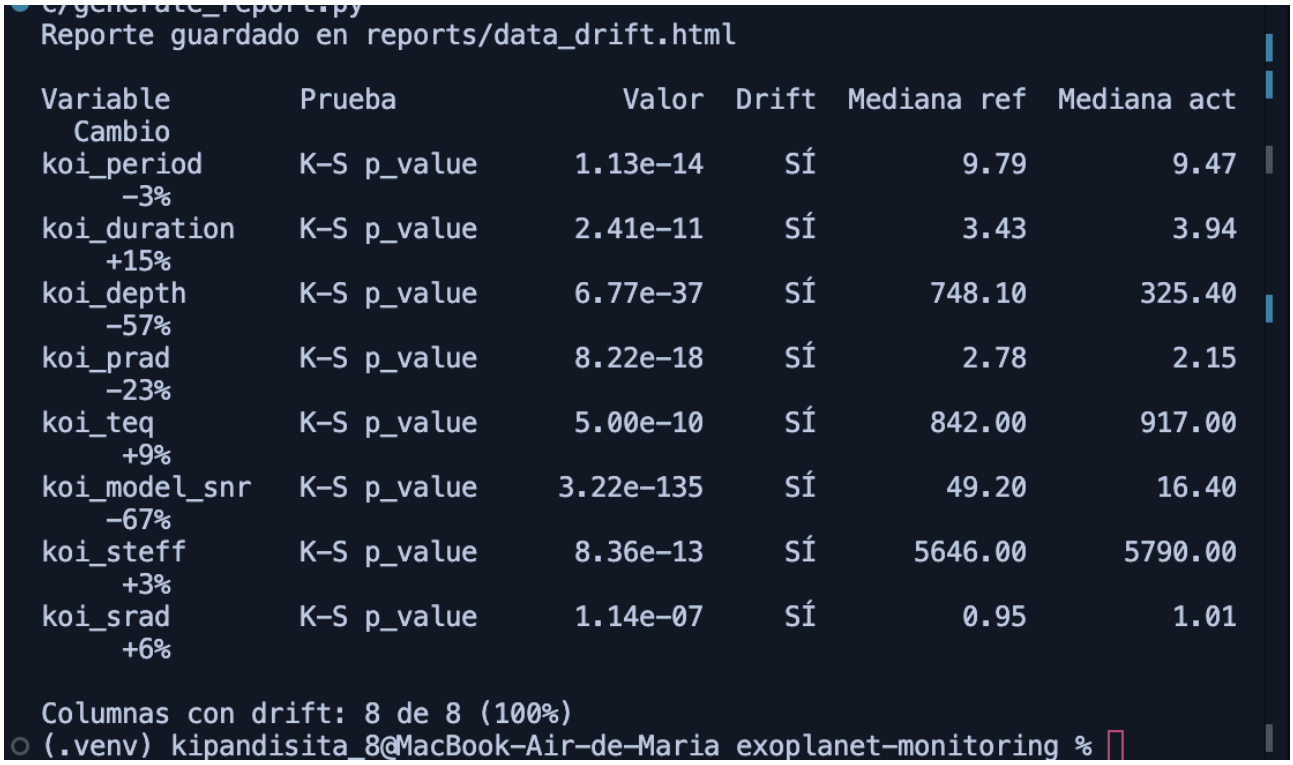
Reporte reports/data_drift.html generado con DataDriftPreset: se detecta drift en el dataset.





Evidencia 3. Variables con drift detectado

Las 8 variables tienen drift según la prueba K-S (p-value < 0.05). La magnitud varía: koi_model_snr -67 % y koi_depth -57 % en la mediana, frente a solo -3 % en koi_period.



```
Reporte guardado en reports/data_drift.html

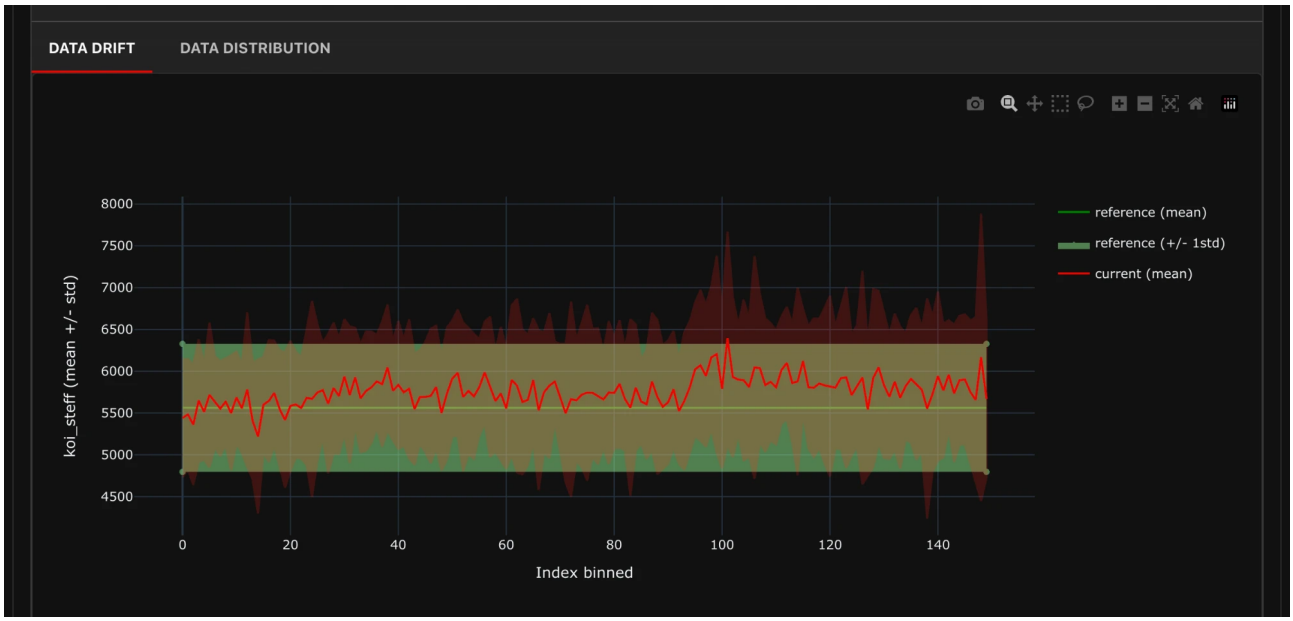
Variable        Prueba              Valor  Drift  Mediana ref  Mediana act
  Cambio
koi_period      K-S p_value      1.13e-14     SÍ         9.79         9.47
     -3%
koi_duration    K-S p_value      2.41e-11     SÍ         3.43         3.94
    +15%
koi_depth       K-S p_value      6.77e-37     SÍ       748.10       325.40
    -57%
koi_prad        K-S p_value      8.22e-18     SÍ         2.78         2.15
    -23%
koi_teq         K-S p_value      5.00e-10     SÍ       842.00       917.00
     +9%
koi_model_snr   K-S p_value     3.22e-135     SÍ        49.20        16.40
    -67%
koi_steff       K-S p_value      8.36e-13     SÍ      5646.00      5790.00
     +3%
koi_srad        K-S p_value      1.14e-07     SÍ         0.95         1.01
     +6%

Columnas con drift: 8 de 8 (100%)
(.venv) kipandisita_8@MacBook-Air-de-Maria exoplanet-monitoring %
```

Evidencia 4. Comparación de distribuciones

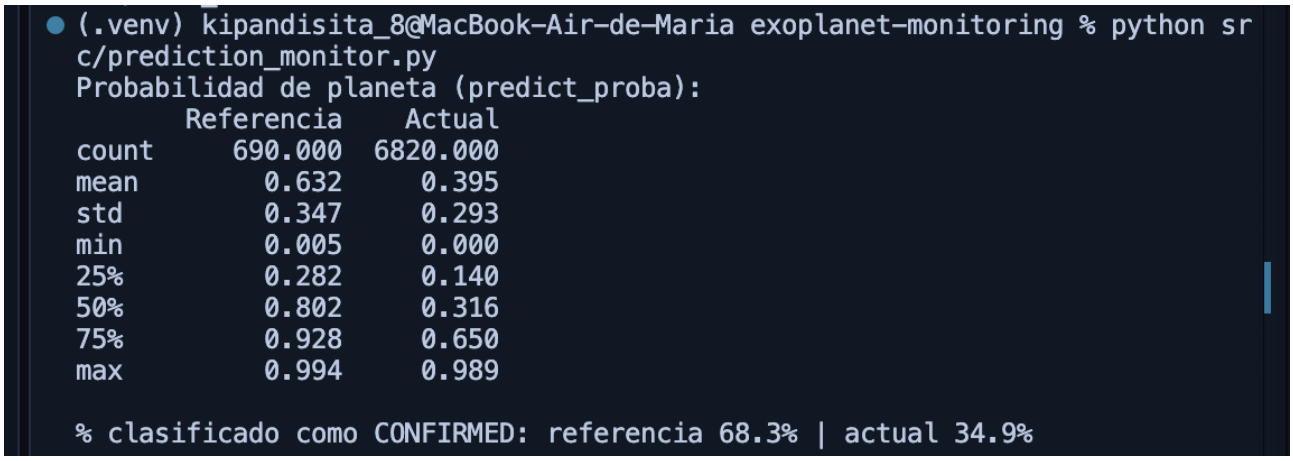
Media de los datos actuales (rojo) frente a la franja de referencia (verde, media ± 1 desviación). En koi_period la media actual se sale de la franja; en koi_steff se desplaza hacia arriba.





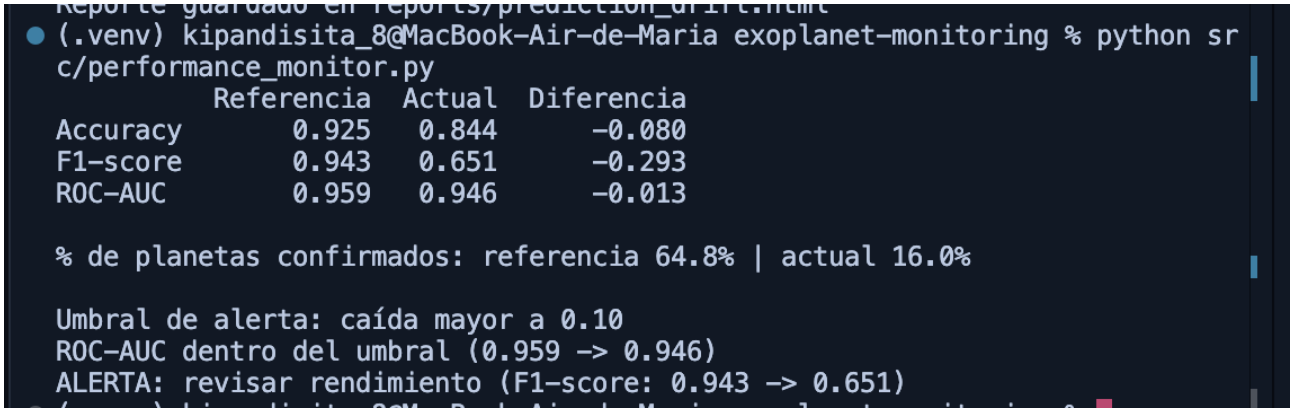
Evidencia 5. Comparación de predicciones

La probabilidad media de planeta baja de 0.632 a 0.395 y los objetos clasificados como CONFIRMED pasan de 68.3 % a 34.9 % (prediction drift).



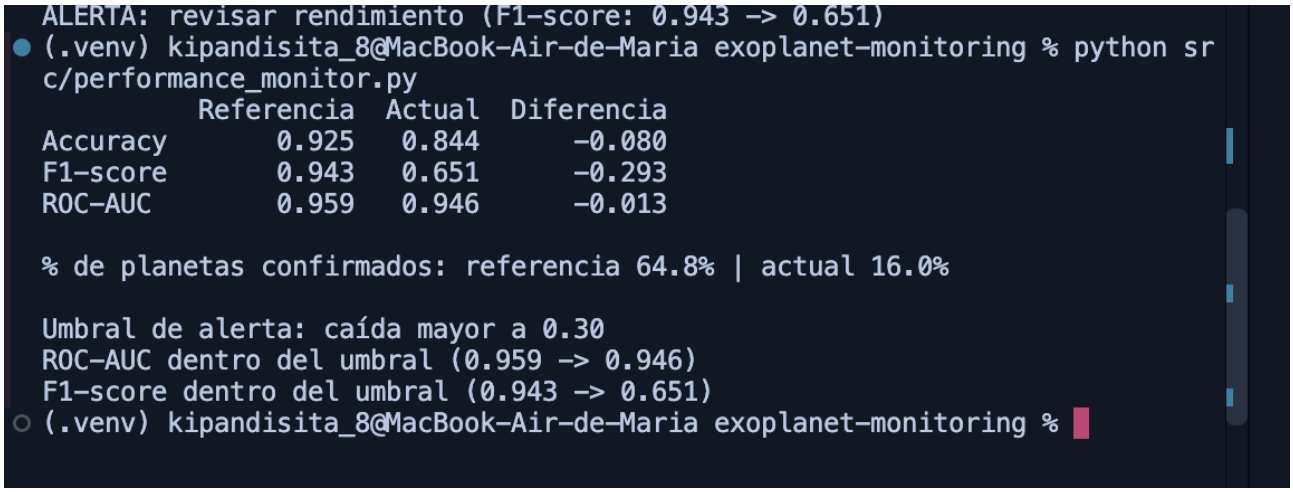
Evidencia 6. Comparación de Accuracy, F1 y ROC-AUC

Con etiquetas reales: Accuracy 0.925 → 0.844, F1 0.943 → 0.651, ROC-AUC 0.959 → 0.946. Con umbral 0.10 se activa la alerta por F1-score.



Evidencia 7. Alerta por deterioro

Con umbral 0.10 (evidencia 6) se activa la ALERTA por F1-score. Al cambiar el umbral a 0.30 la caída de F1 (0.293) queda dentro del umbral y ya no hay alerta: el resultado depende del umbral elegido.



Evidencia 8. Interpretación de los resultados y decisión recomendada

Contexto

El modelo Champion (Random Forest) se entrenó con los primeros objetos descubiertos por Kepler (KOI < 2000). Después "recibe" los objetos descubiertos más tarde (KOI >= 2000). Comparamos ambos periodos con Evidently AI.

Data drift

Las 8 variables presentan drift según la prueba K-S (p-value < 0.05). Sin embargo, la magnitud del cambio es muy distinta:

Variable	Mediana ref.	Mediana actual	Cambio	Interpretación
koi_model_snr	49.2	16.4	-67 %	Las señales nuevas son mucho más débiles
koi_depth	748.1	325.4	-57 %	Tránsitos menos profundos: objetos más pequeños o más difíciles de detectar
koi_prad	2.78	2.15	-23 %	Planetas candidatos más pequeños
koi_duration	3.43	3.94	+15 %	Tránsitos algo más largos
koi_period	9.79	9.47	-3 %	Drift estadístico, pero sin relevancia práctica

Con miles de registros, la prueba K-S detecta incluso cambios pequeños. Por eso el drift debe interpretarse junto con su magnitud: `koi_period` cambia muy poco, mientras que `koi_model_snr` y `koi_depth` cambian de forma importante.

Esto tiene sentido físico: al inicio de la misión se detectaron primero los planetas más fáciles (grandes y con señal fuerte). Con más datos, Kepler pudo detectar señales más débiles, y entre ellas hay más falsos positivos.

Prediction drift

La probabilidad media de "planeta confirmado" bajó de 0.632 a 0.395, y el porcentaje de objetos clasificados como CONFIRMED bajó de 68.3 % a 34.9 %.

Esto **no significa por sí solo que el modelo esté fallando**: la población cambió. En los datos actuales solo el 16 % de los objetos con etiqueta son planetas confirmados, frente al 64.8 % en la referencia. Que el modelo prediga menos planetas es, en parte, una respuesta correcta a ese cambio.

Rendimiento (con etiquetas reales)

Métrica	Referencia	Actual	Diferencia
Accuracy	0.925	0.844	-0.080
F1-score	0.943	0.651	-0.293
ROC-AUC	0.959	0.946	-0.013

• **ROC-AUC casi no cambió**: el modelo sigue ordenando bien los objetos (los planetas reales siguen recibiendo probabilidades más altas que los falsos positivos).

• **F1 cayó fuertemente**: con el umbral fijo de 0.5, el modelo marca como planeta a muchos falsos positivos. Como ahora los planetas son minoría (16 %), esos errores pesan mucho más en la precisión y por tanto en el F1.

Data drift, prediction drift y concept drift

Tipo	¿Qué cambia?	¿Se observa aquí?	Evidencia
Data drift	La distribución de las variables de entrada	Sí, fuerte	8 de 8 variables; koi_model_snr -67 %, koi_depth -57 %
Prediction drift	La distribución de las predicciones	Sí	Probabilidad media 0.632 → 0.395
Concept drift	La relación entre las variables y el resultado real	No hay evidencia fuerte	ROC-AUC casi igual (0.959 → 0.946)
Performance degradation	El rendimiento predictivo	Sí, en F1	F1 0.943 → 0.651

El **concept drift** solo puede evaluarse con etiquetas reales. Si la relación entre las variables físicas y "ser planeta" hubiera cambiado (por ejemplo, que una señal fuerte ya no indicara un planeta), el modelo ordenaría peor los objetos y el ROC-AUC caería. Como el ROC-AUC se mantiene, la relación parece estable: el modelo sigue distinguiendo bien planetas de falsos positivos.

Lo que sí cambió es la **proporción de clases** (de 64.8 % a 16 % de planetas). Eso no es concept drift, sino un cambio en la población que afecta al umbral de decisión de 0.5, y por eso cae el F1 aunque el ROC-AUC no.

Alerta

Con un umbral de caída de 0.10, la regla **no se activa con ROC-AUC** (-0.013) pero **sí con F1-score** (-0.293). La elección de la métrica para la alerta es importante: si solo vigiláramos ROC-AUC, no detectaríamos este problema.

Al subir el umbral a 0.30, la caída de F1 (0.293) queda dentro del umbral y la alerta desaparece. Esto muestra que el umbral debe justificarse según el problema: uno demasiado alto puede ocultar un deterioro real.

Nota sobre la Actividad 7: la guía propone escribir a mano `auc_reference = 0.85` y `auc_current = 0.71` y luego cambiar `auc_current` a 0.82. En este proyecto las métricas se calculan automáticamente a partir de las predicciones y las etiquetas reales, como recomienda la propia guía para una implementación real. Por eso, para ver cómo cambia el resultado, se modificó el umbral (0.10 → 0.30) en lugar de un valor escrito a mano. La lógica de la alerta es la misma: comparar la caída contra un umbral.

Análisis de escenarios

Situación	Acción recomendada	¿Aplica a este caso?
No hay drift y rendimiento estable	Continuar monitoreando	No: hay drift en las 8 variables
Hay data drift, pero rendimiento estable	Investigar el cambio y mantener vigilancia	Parcialmente: ROC-AUC estable, pero F1 no
Hay prediction drift sin etiquetas reales	Investigar y esperar etiquetas	Fue el primer paso: con current.csv solo veíamos el cambio en las predicciones
El rendimiento disminuye significativamente	Revisar datos, modelo y posibles causas	**Sí**: F1 cae 0.293 y se activa la alerta
El rendimiento disminuye de forma persistente	Evaluar reentrenamiento y validarlo	Aún no se sabe: solo tenemos un periodo actual

El orden del monitoreo fue el de la tabla: primero vimos el data drift y el prediction drift sin etiquetas; al llegar las etiquetas reales (current_with_labels.csv) confirmamos una caída significativa de F1. Para decir que la caída es **persistente** necesitaríamos varios periodos nuevos con el mismo comportamiento.

Decisión recomendada

Nuestro caso corresponde a "el rendimiento disminuye significativamente", por lo que hay que **revisar datos, modelo y posibles causas** antes de actuar:

1. La causa principal parece ser un cambio en la población (más falsos positivos y señales más débiles), no un error en el código.

2. Como ROC-AUC se mantiene, una primera medida barata sería **recalibrar el umbral de decisión** con datos recientes etiquetados.

3. Si el deterioro persiste, **evaluar un reentrenamiento** que incluya datos del periodo actual, validándolo con un conjunto separado antes de desplegarlo.

Detectar drift no implica reentrenar automáticamente: la decisión se justifica con evidencia y validación.

Kubernetes: autoescalado de la API de exoplanetas

Docker Desktop Kubernetes + metrics-server + Horizontal Pod Autoscaler

Autora: Mafer González

Repositorio: https://github.com/Mafersita-code/exoplanet-cicd/tree/main/k8s

Objetivo: simular tráfico elevado contra la API FastAPI de exoplanetas, observar cómo un solo contenedor se satura y cómo Kubernetes resuelve el problema creando réplicas automáticamente con la misma imagen Docker.

Resultados (50 usuarios simultáneos)

	1 réplica	Con HPA (1 a 5 réplicas)
Peticiones por segundo	~20-32	~70-100 (picos de 103)
Latencia mediana (p50)	~1,600-2,000 ms	~100-500 ms
Errores	0 %	0.3 % (al arrancar réplicas)
Réplicas	1 fija	1 -> 4 -> 5 -> 1

Evidencias

N.º	Evidencia
1	Manifiestos de Kubernetes
2	Clúster funcionando
3	Una sola réplica bajo carga
4	Activación del autoescalado
5	El HPA escala y desescala
6	Réplicas nuevas creadas
7	Rendimiento con autoescalado
8	Manifiestos publicados en GitHub

Evidencia 1. Manifiestos de Kubernetes

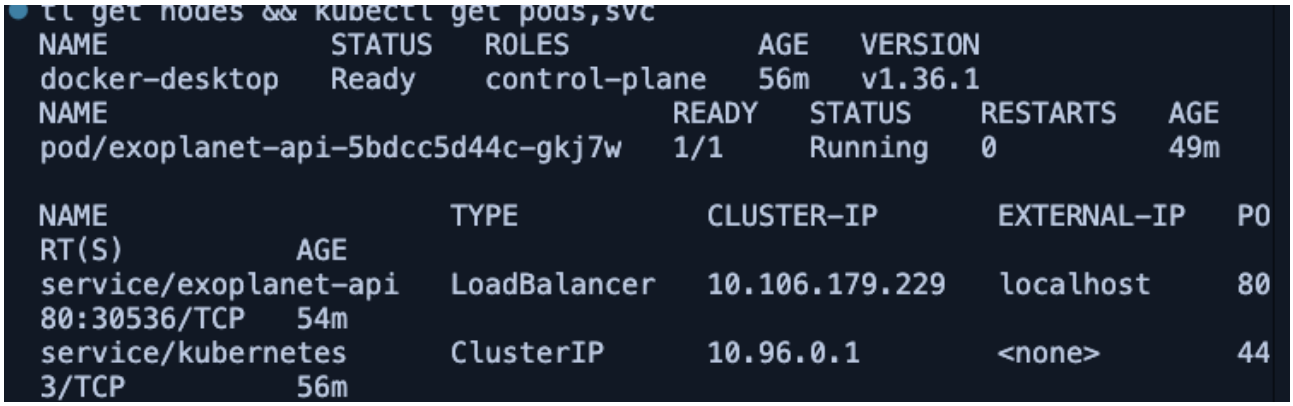
deployment.yaml corre la imagen exoplanet-api:v1 con 1 réplica, límites de CPU (500m) y memoria, y probes de salud. hpa.yaml define el autoescalado: de 1 a 5 réplicas cuando el CPU promedio supera el 50 % del request.





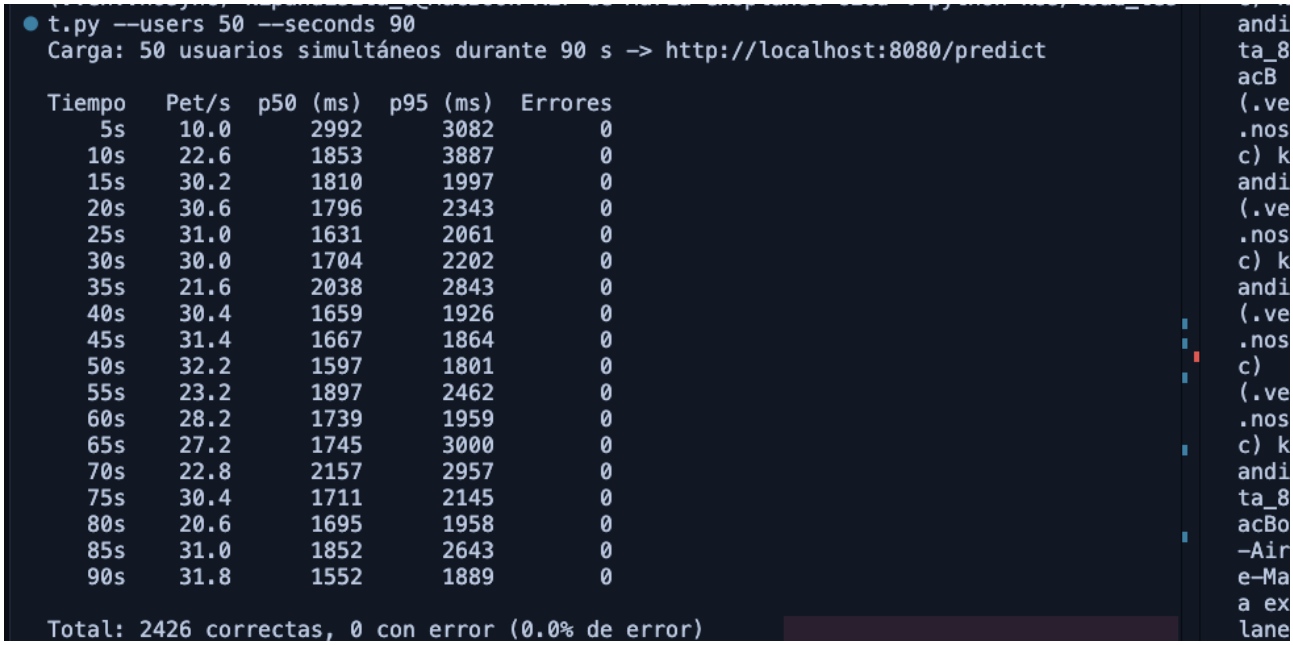
Evidencia 2. Clúster funcionando

Nodo docker-desktop en estado Ready, 1 pod de la API en Running y el Service tipo LoadBalancer expuesto en localhost:8080.



Evidencia 3. Una sola réplica bajo carga

50 usuarios simultáneos contra POST /predict durante 90 s. Con un solo contenedor la API se satura: ~20-32 peticiones/s y una latencia mediana de ~1,600-2,000 ms por predicción.



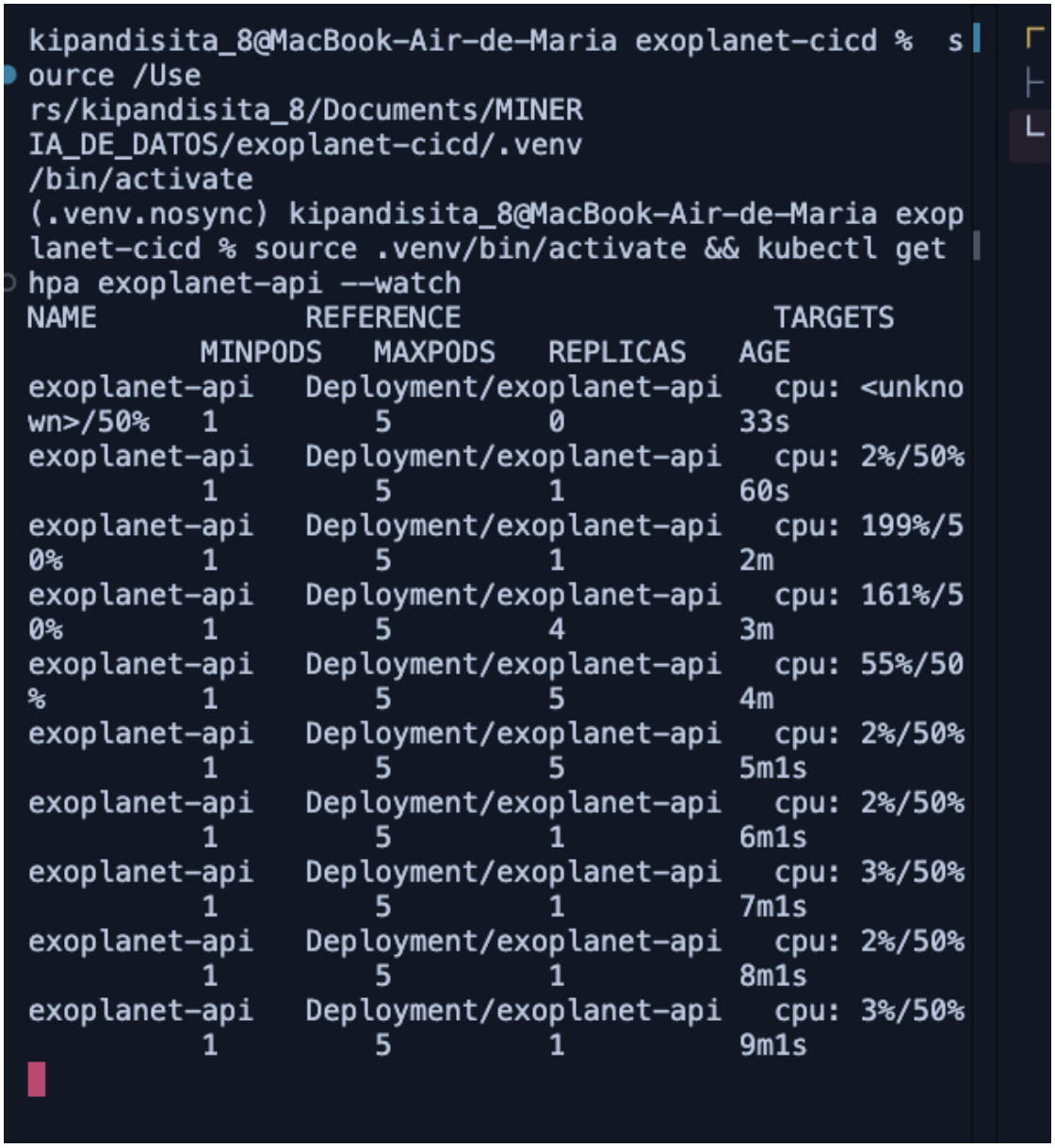
Evidencia 4. Activación del autoescalado

Se crea el HorizontalPodAutoscaler a partir de hpa.yaml.



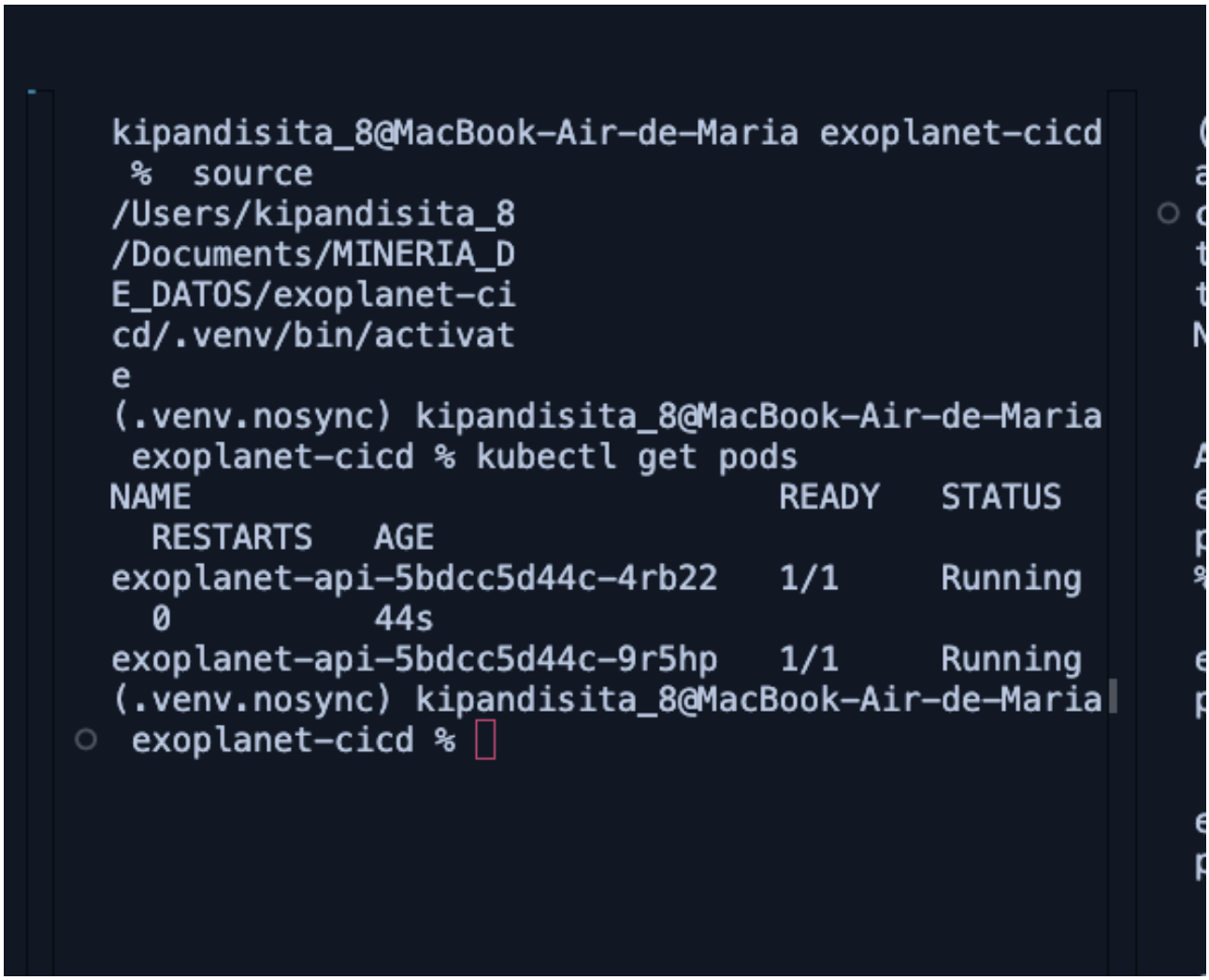
Evidencia 5. El HPA escala y desescala

Al llegar la carga el CPU sube a 199 % del request (objetivo 50 %) y el HPA aumenta las réplicas de 1 a 4 y luego a 5. Cuando la carga termina, el CPU baja a ~2 % y, tras la ventana de estabilización de 60 s, el HPA regresa a 1 réplica.



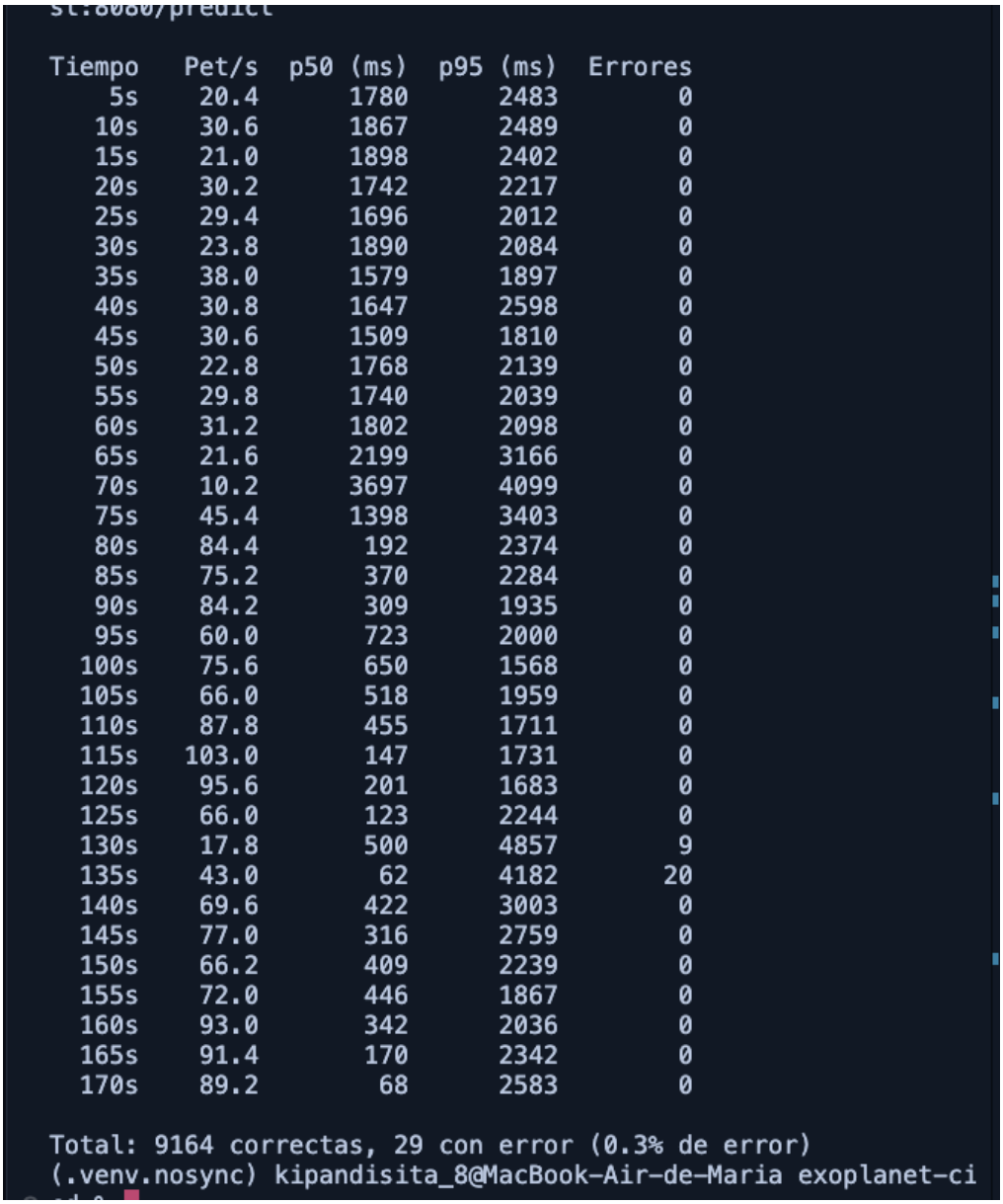
Evidencia 6. Réplicas nuevas creadas

Pods nuevos creados automáticamente por el HPA con la misma imagen (44 s de antigüedad), en estado Running y listos para recibir tráfico.



Evidencia 7. Rendimiento con autoescalado

Misma carga de 50 usuarios durante 170 s. Los primeros ~70 s se comportan como con 1 réplica; en cuanto entran las réplicas nuevas, las peticiones/s suben a ~70-100 y la latencia mediana baja a ~100-500 ms. Total: 9,164 correctas y solo 0.3 % de errores (durante el arranque de réplicas).



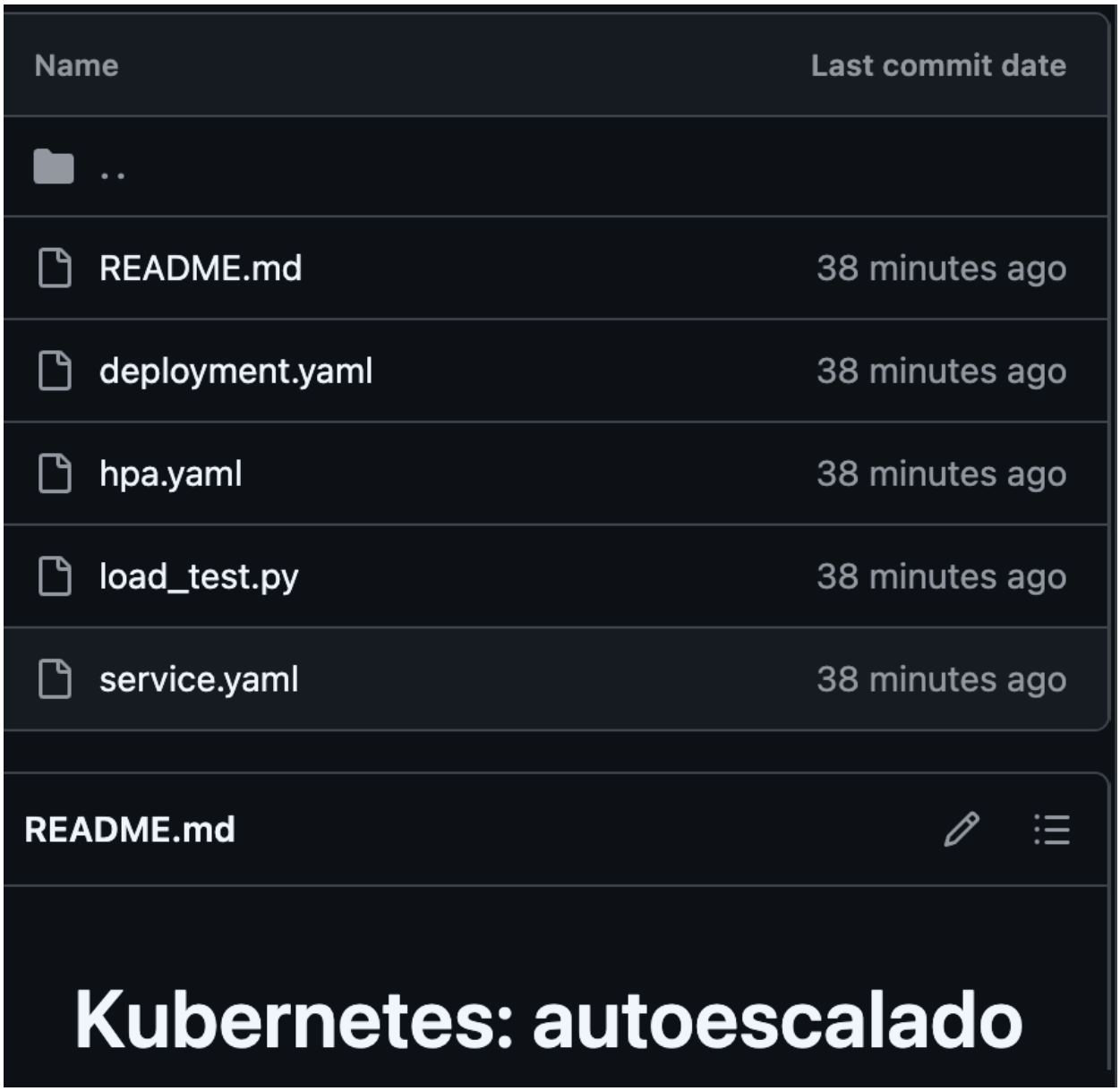
```
:8080/predict

Tiempo   Pet/s  p50 (ms)  p95 (ms)  Errores
    5s    20.4      1780      2483        0
   10s    30.6      1867      2489        0
   15s    21.0      1898      2402        0
   20s    30.2      1742      2217        0
   25s    29.4      1696      2012        0
   30s    23.8      1890      2084        0
   35s    38.0      1579      1897        0
   40s    30.8      1647      2598        0
   45s    30.6      1509      1810        0
   50s    22.8      1768      2139        0
   55s    29.8      1740      2039        0
   60s    31.2      1802      2098        0
   65s    21.6      2199      3166        0
   70s    10.2      3697      4099        0
   75s    45.4      1398      3403        0
   80s    84.4       192      2374        0
   85s    75.2       370      2284        0
   90s    84.2       309      1935        0
   95s    60.0       723      2000        0
  100s    75.6       650      1568        0
  105s    66.0       518      1959        0
  110s    87.8       455      1711        0
  115s   103.0       147      1731        0
  120s    95.6       201      1683        0
  125s    66.0       123      2244        0
  130s    17.8       500      4857        9
  135s    43.0        62      4182       20
  140s    69.6       422      3003        0
  145s    77.0       316      2759        0
  150s    66.2       409      2239        0
  155s    72.0       446      1867        0
  160s    93.0       342      2036        0
  165s    91.4       170      2342        0
  170s    89.2        68      2583        0

Total: 9164 correctas, 29 con error (0.3% de error)
(.venv.nosync) kipandisita_8@MacBook-Air-de-Maria exoplanet-ci
```

Evidencia 8. Manifiestos publicados en GitHub

La carpeta k8s está en el repositorio del proyecto:
https://github.com/Mafersita-code/exoplanet-cicd/tree/main/k8s



Análisis

¿Por qué se satura un solo contenedor? Cada predicción del Random Forest consume CPU. Con el límite de 0.5 CPU, un solo pod procesa unas 30 peticiones por segundo; con 50 usuarios enviando sin parar, las peticiones hacen fila y cada una tarda ~2 s en responder.

¿Cómo lo resuelve Kubernetes? metrics-server mide el CPU de cada pod. El HPA compara el promedio con el objetivo (50 % del request de 250m) y calcula cuántas réplicas necesita. Al ver 199 % creó réplicas nuevas con la misma imagen, y el Service repartió el tráfico entre ellas. Con 5 réplicas la capacidad se multiplicó por ~3 y la latencia bajó de segundos a milisegundos.

¿Por qué no es instantáneo? El HPA revisa las métricas cada ~15 s y cada pod nuevo necesita unos segundos para arrancar y pasar su readiness probe. Por eso los primeros ~70 s de la prueba se parecen a la de una sola réplica, y los pocos errores (0.3 %) aparecen mientras entran los pods.

Desescalado. Al terminar la carga el CPU bajó a ~2 % y el HPA regresó a 1 réplica tras la ventana de estabilización de 60 s, liberando recursos.

Lección aprendida: la readiness probe

En la primera prueba la readiness probe usaba el tiempo de espera por defecto de 1 s. Bajo carga, /health tardaba más que eso, Kubernetes marcaba el pod como "no listo" y lo sacaba del Service: la API dejó de responder (36 % de errores). Además, el HPA ignora las métricas de los pods no listos, así que tampoco escalaba. Con `timeoutSeconds: 5` el pod sigue recibiendo tráfico aunque esté lento y el HPA puede medirlo y escalar. Una configuración de salud demasiado estricta puede provocar justamente la caída que se quería evitar.

AWS: despliegue del flujo MLOps de exoplanetas

Enfoque de servicios individuales: S3 + DVC + ECR + ECS/Fargate + CloudWatch + GitHub Actions + MLflow + Airflow

Autora: Mafer González

Repositorio: https://github.com/Mafersita-code/exoplanet-cicd

Objetivo: llevar a la nube de AWS el flujo construido en las prácticas anteriores (Git, DVC, MLflow, Airflow, FastAPI, Docker y CI/CD): guardar datos y modelos en S3, publicar la imagen Docker en ECR, ejecutar la API en ECS con Fargate, monitorearla con CloudWatch automatizar la publicación de la imagen con GitHub Actions y orquestar el entrenamiento con Airflow. En una segunda etapa, MLflow y Airflow se movieron de la computadora local a un servidor EC2, para que todo el flujo corra en la nube.

Región: us-east-2 (Ohio). **Cuenta:** AWS Free Plan, con un usuario IAM propio (*mafer-mlops*) para la CLI.

Arquitectura

Servicio	Uso en el proyecto
Amazon S3	Bucket *mafer-exoplanetas-mlops-426143721250*: dataset, modelo, remote de DVC y artefactos de MLflow.
DVC	Versiona *data/cumulative.csv*; el archivo real vive en S3 y en Git solo queda *cumulative.csv.dvc*.
Amazon ECR	Repositorio privado *exoplanet-api* con la imagen Docker de la API (linux/amd64).
Amazon ECS + Fargate	Clúster *exoplanet-cluster* y servicio *exoplanet-service*: corre el contenedor sin administrar servidores.
CloudWatch	Log group */ecs/exoplanet-api* con los logs de la API; supervisión del clúster.
GitHub Actions	CI (pytest) y CD: en cada push a main construye la imagen y la publica en ECR.
MLflow	Registra parámetros, métricas y el modelo *exoplanet-champion*; corre en EC2 y los artefactos se guardan en S3.
Apache Airflow	DAG *exoplanet_pipeline* en EC2: datos desde S3 (DVC) → entrenamiento (MLflow) → pruebas → modelo a S3.
Amazon EC2 + IAM	Servidor *mlops-server* (m7i-flex.large, Ubuntu 24.04) con MLflow y Airflow como servicios de systemd; un rol IAM le da acceso solo al bucket del proyecto.

Evidencias

N.º	Evidencia
1	Datos y modelo en Amazon S3
2	Imagen Docker publicada en Amazon ECR
3	Clúster de ECS con el servicio activo
4	Tarea en Fargate con IP pública
5	API funcionando en AWS
6	Monitoreo con CloudWatch
7	CI/CD con GitHub Actions
8	Imágenes publicadas automáticamente en ECR
9	Entrenamientos registrados en MLflow
10	Pipeline orquestado con Airflow (local)
11	Verificación de recursos antes de desplegar
12	Servidor EC2 para MLflow y Airflow
13	Entrenamiento registrado en MLflow desde EC2
14	Artefactos de MLflow en S3

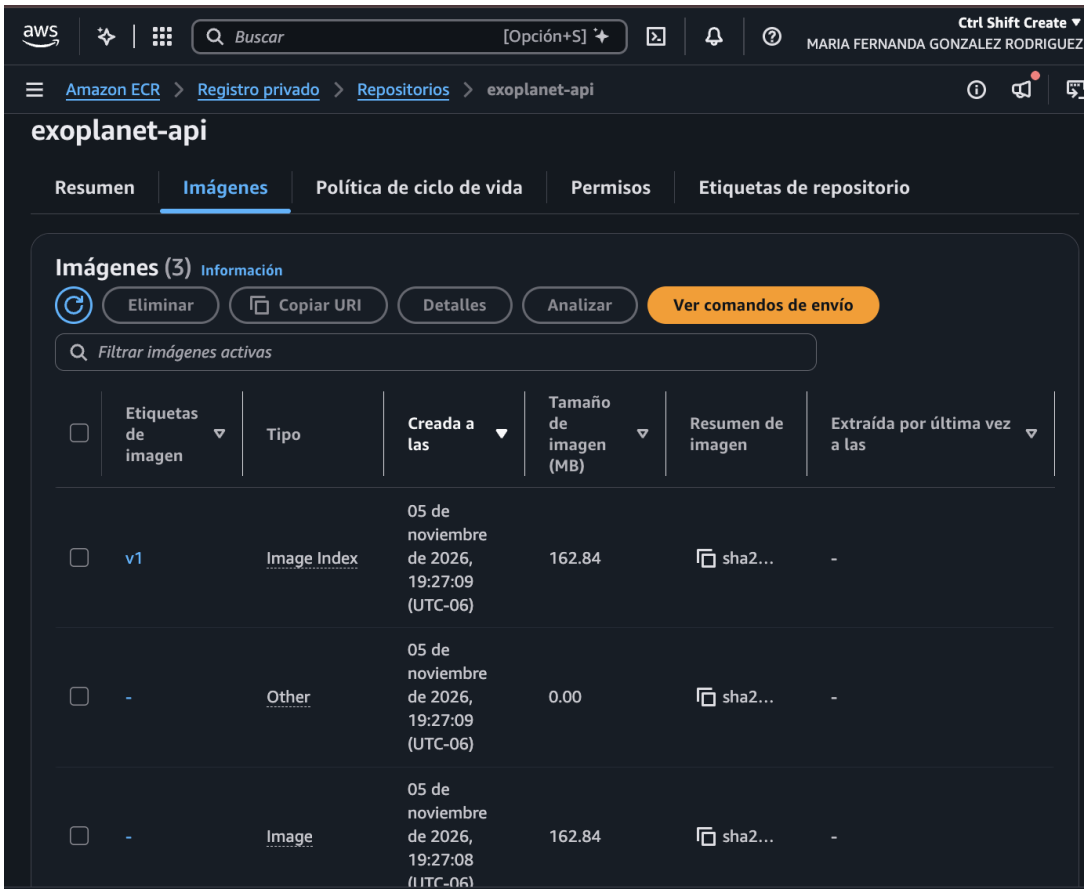
Evidencia 1. Datos y modelo en Amazon S3

Se crea el bucket con *aws s3 mb* y se suben el dataset (*data/cumulative.csv*, 3.6 MB) y el modelo (*models/champion.joblib*, 2.0 MB). El listado final confirma que ambos archivos están en la nube. Después, el mismo bucket se configuró como remote de DVC (*s3://…/dvc*) para versionar el dataset.

```
kipandisita_8@MacBook-Air-de-Maria exoplanet-cicd % aws s3 mb s3://$BUCKET --region $REGION
aws s3 cp data/cumulative.csv s3://$BUCKET/data/cumulative.csv
aws s3 cp models/champion.joblib s3://$BUCKET/models/champion.joblib
aws s3 ls s3://$BUCKET --recursive
make_bucket: mafer-exoplanetas-mlops-426143721250
upload: data/cumulative.csv to s3://mafer-exoplanetas-mlops-426143721250/data/cumulative.csv
upload: models/champion.joblib to s3://mafer-exoplanetas-mlops-426143721250/models/champion.joblib
2026-11-05 19:24:52    3695322 data/cumulative.csv
2026-11-05 19:24:55    2083506 models/champion.joblib
kipandisita_8@MacBook-Air-de-Maria exoplanet-cicd %
```

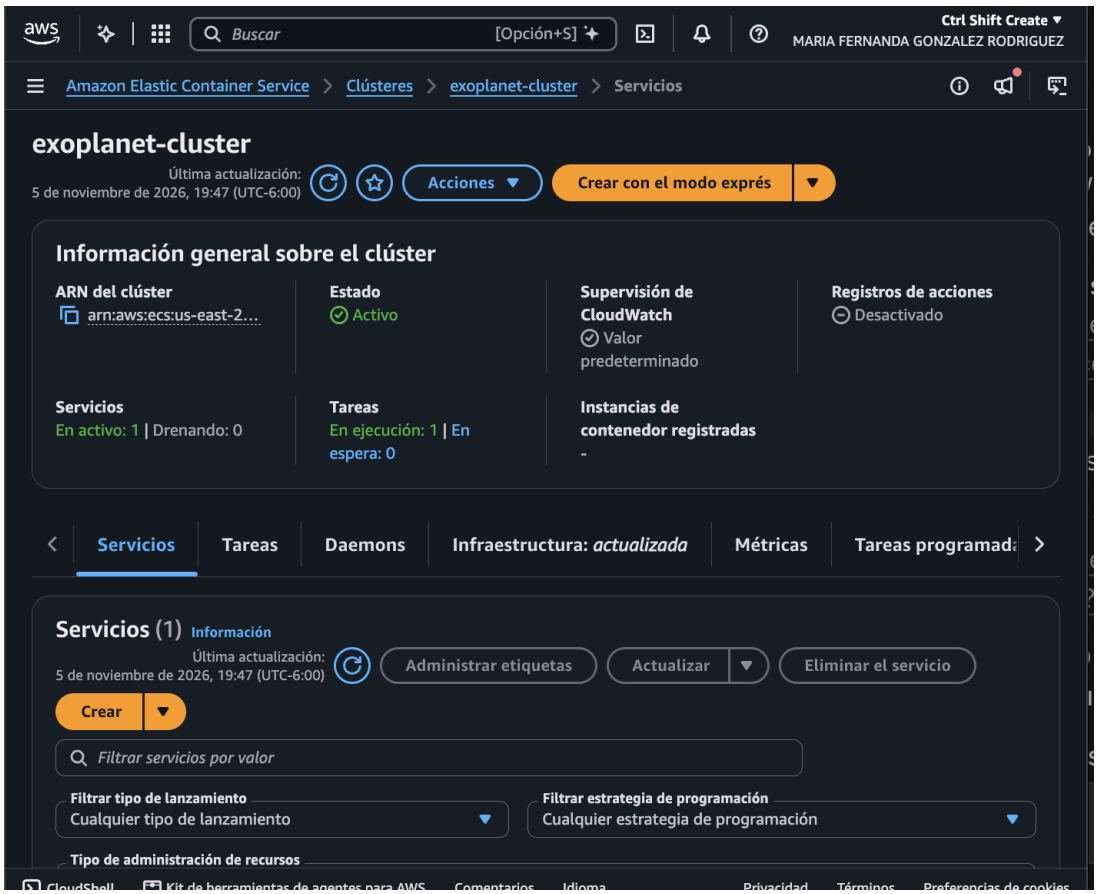
Evidencia 2. Imagen Docker publicada en Amazon ECR

Repositorio privado *exoplanet-api* con la imagen **v1** (162.84 MB). Se construyó con *docker buildx --platform linux/amd64*, porque la Mac es ARM y Fargate corre en x86.



Evidencia 3. Clúster de ECS con el servicio activo

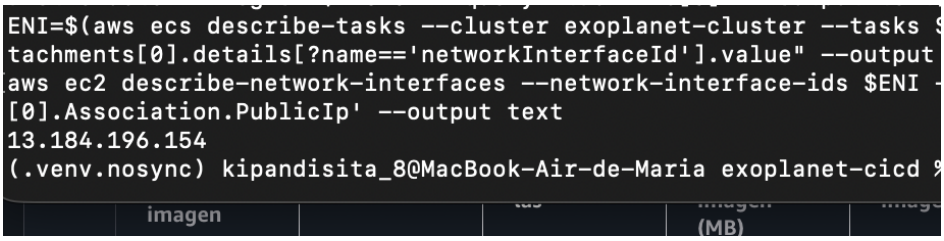
El clúster *exoplanet-cluster* está **Activo**, con 1 servicio en activo, 1 tarea en ejecución y la supervisión de CloudWatch habilitada.



Evidencia 4. Tarea en Fargate con IP pública

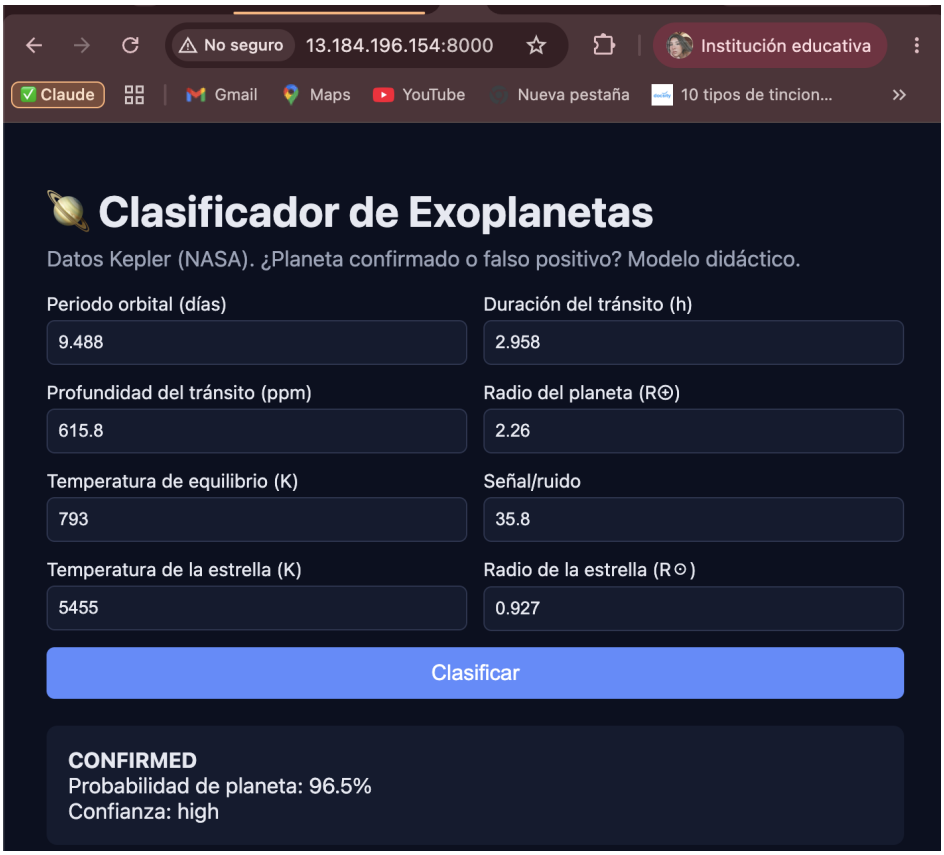
La tarea corre en modo de red *awsvpc* dentro de la VPC por defecto, con el grupo de seguridad *exoplanet-sg* (puerto 8000 abierto) y la IP pública **13.184.196.154**, obtenida también desde la CLI.

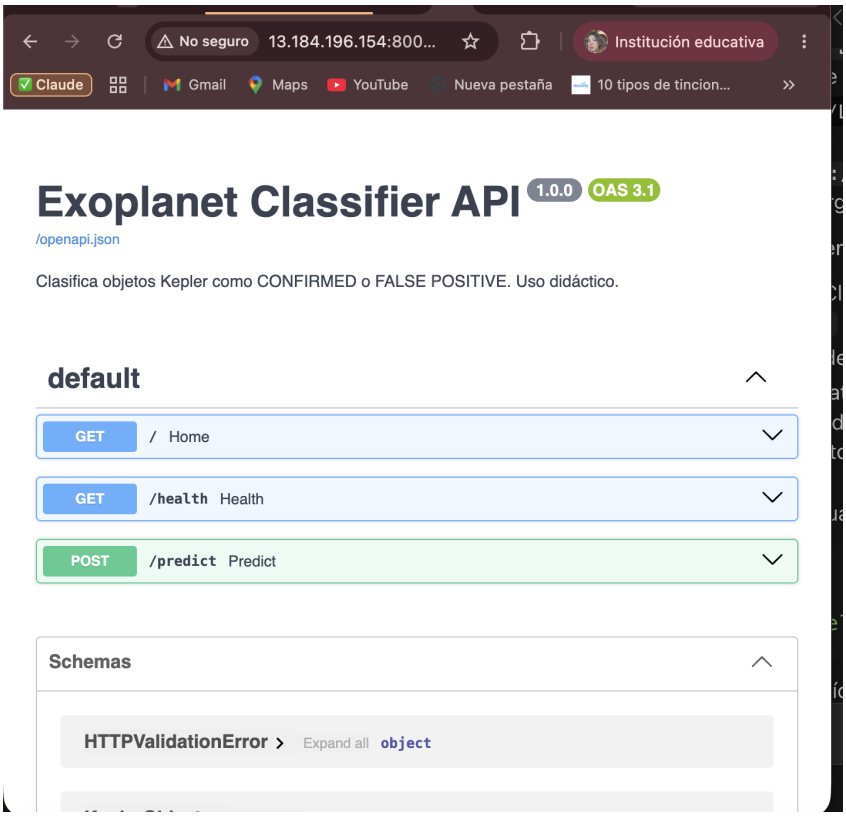


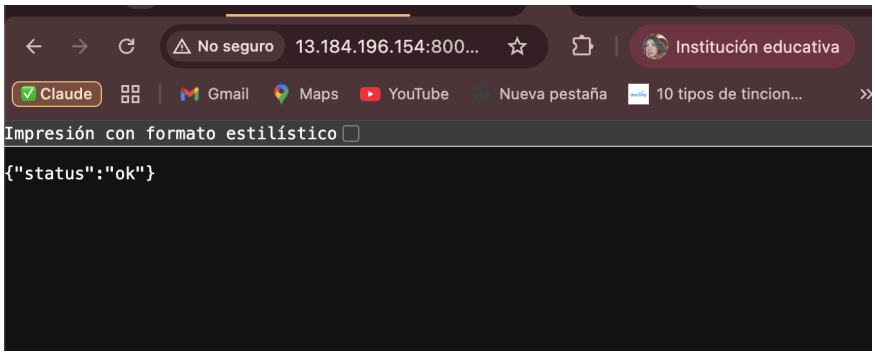


Evidencia 5. API funcionando en AWS

El formulario de la API, servido desde Fargate en *http://13.184.196.154:8000*, clasifica Kepler-227 b como **CONFIRMED** con 96.5 % de probabilidad y confianza alta. La documentación interactiva */docs* muestra los endpoints */*, */health* y */predict*, y */health* responde *{"status":"ok"}*.

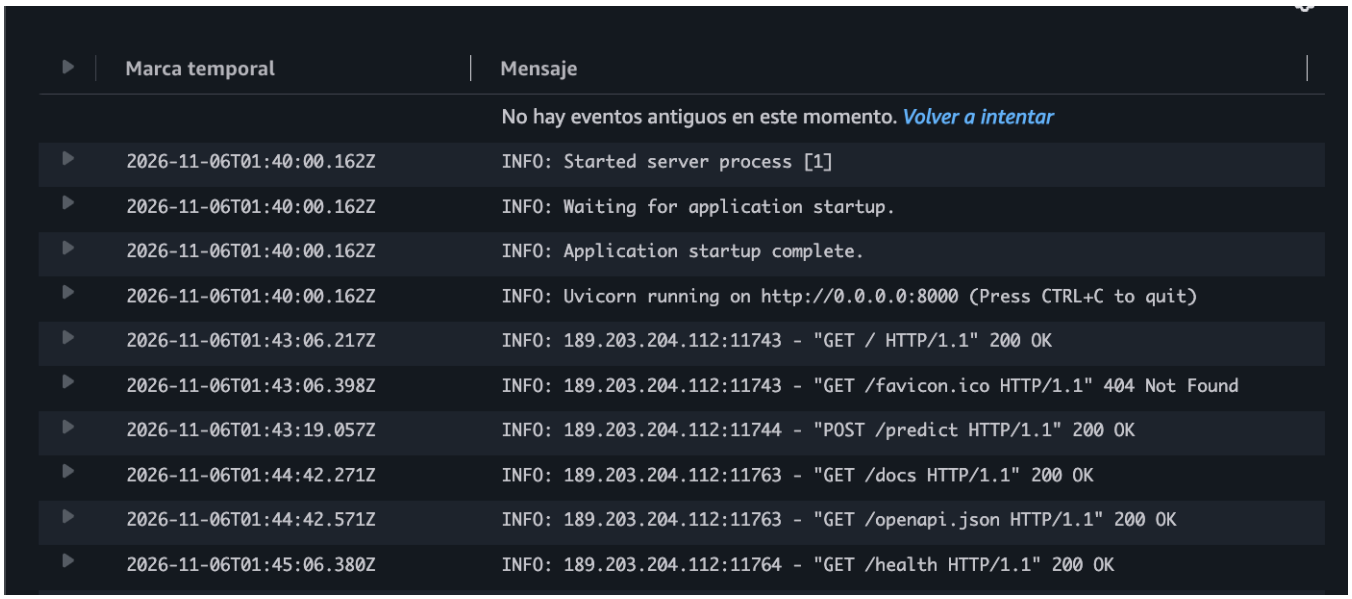



No seguro
13.184.196.154:800...
Institución educativa
Claude
Gmail
Maps
YouTube
Nueva pestaña
10 tipos de tincion...
Exoplanet Classifier API
1.0.0
OAS 3.1
/openapi.json
Clasifica objetos Kepler como CONFIRMED o FALSE POSITIVE. Uso didáctico.
default
GET / Home
GET /health Health
POST /predict Predict
Schemas
HTTPValidationError
Expand all
object


No seguro
13.184.196.154:800...
Institución educativa
Claude
Gmail
Maps
YouTube
Nueva pestaña
10 tipos de tincion...
Impresión con formato estilístico
{"status":"ok"}

Evidencia 6. Monitoreo con CloudWatch

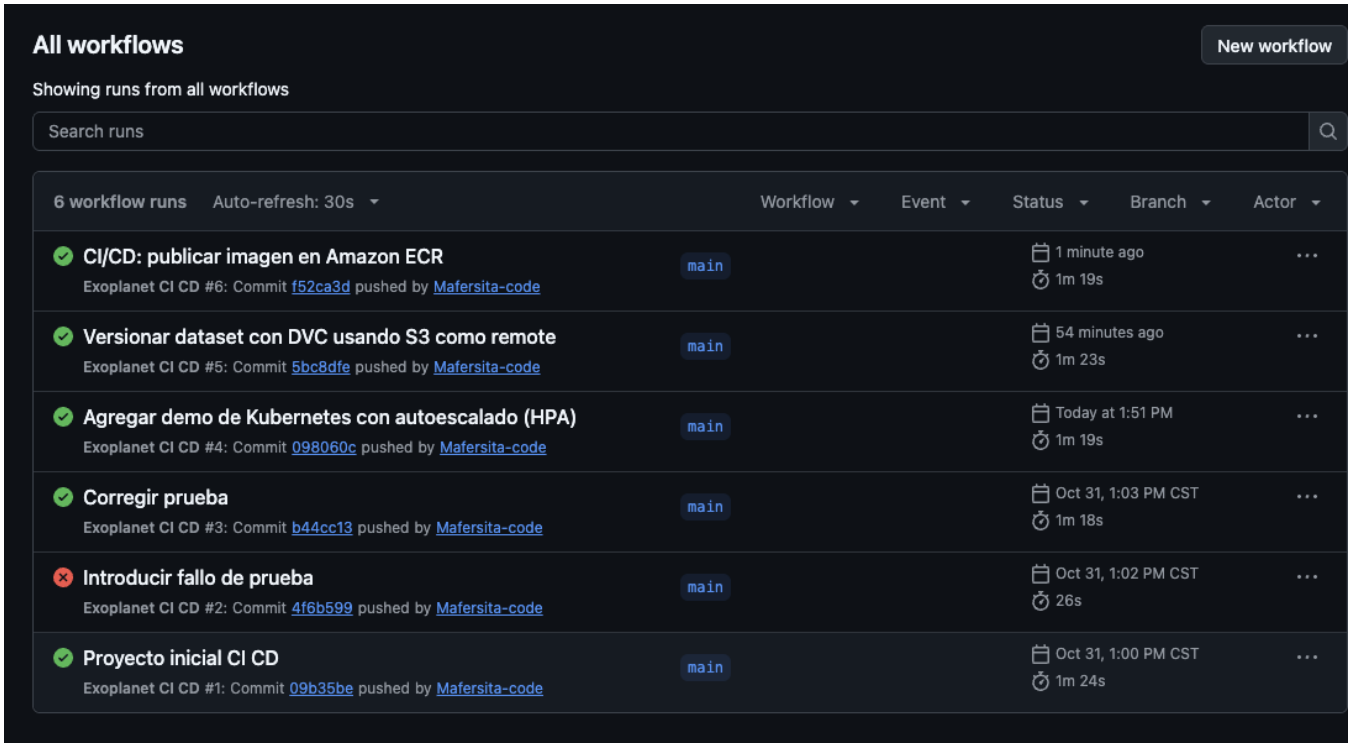
El log group */ecs/exoplanet-api* recibe en tiempo real los logs del contenedor: el arranque de uvicorn y cada petición a la API (*GET /*, *POST /predict*, *GET /docs*, *GET /health*) con su código de respuesta 200 OK.



Marca temporal	Mensaje
	No hay eventos antiguos en este momento. *Volver a intentar*
2026-11-06T01:40:00.162Z	INFO: Started server process [1]
2026-11-06T01:40:00.162Z	INFO: Waiting for application startup.
2026-11-06T01:40:00.162Z	INFO: Application startup complete.
2026-11-06T01:40:00.162Z	INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
2026-11-06T01:43:06.217Z	INFO: 189.203.204.112:11743 - "GET / HTTP/1.1" 200 OK
2026-11-06T01:43:06.398Z	INFO: 189.203.204.112:11743 - "GET /favicon.ico HTTP/1.1" 404 Not Found
2026-11-06T01:43:19.057Z	INFO: 189.203.204.112:11744 - "POST /predict HTTP/1.1" 200 OK
2026-11-06T01:44:42.271Z	INFO: 189.203.204.112:11763 - "GET /docs HTTP/1.1" 200 OK
2026-11-06T01:44:42.571Z	INFO: 189.203.204.112:11763 - "GET /openapi.json HTTP/1.1" 200 OK
2026-11-06T01:45:06.380Z	INFO: 189.203.204.112:11764 - "GET /health HTTP/1.1" 200 OK

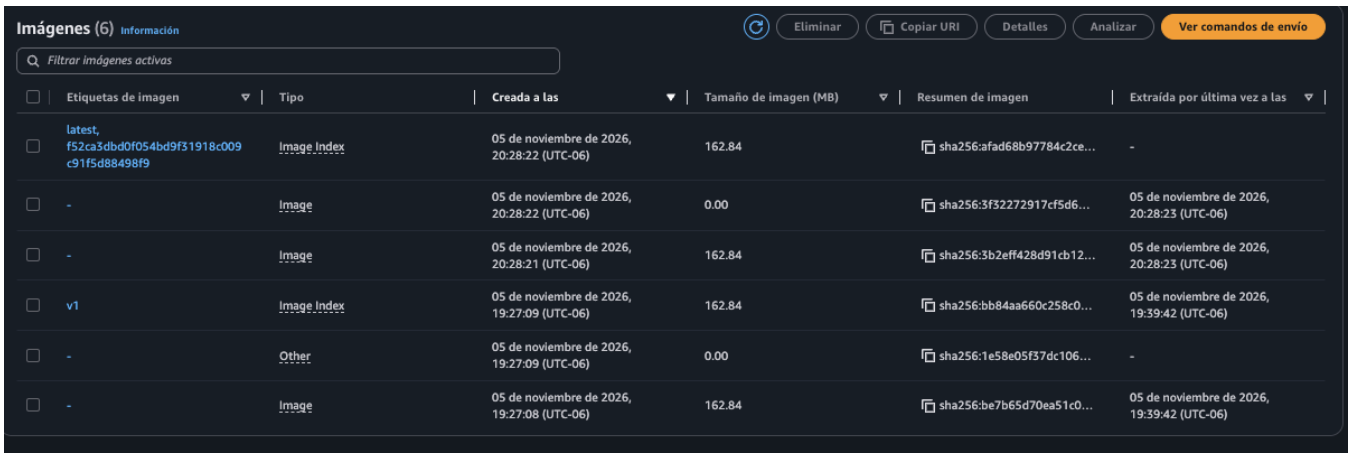
Evidencia 7. CI/CD con GitHub Actions

El workflow ahora tiene dos jobs: **test** (pytest) y **docker**, que inicia sesión en AWS con las access keys guardadas como *secrets* del repositorio y publica la imagen en ECR. La ejecución *CI/CD: publicar imagen en Amazon ECR* terminó en verde.



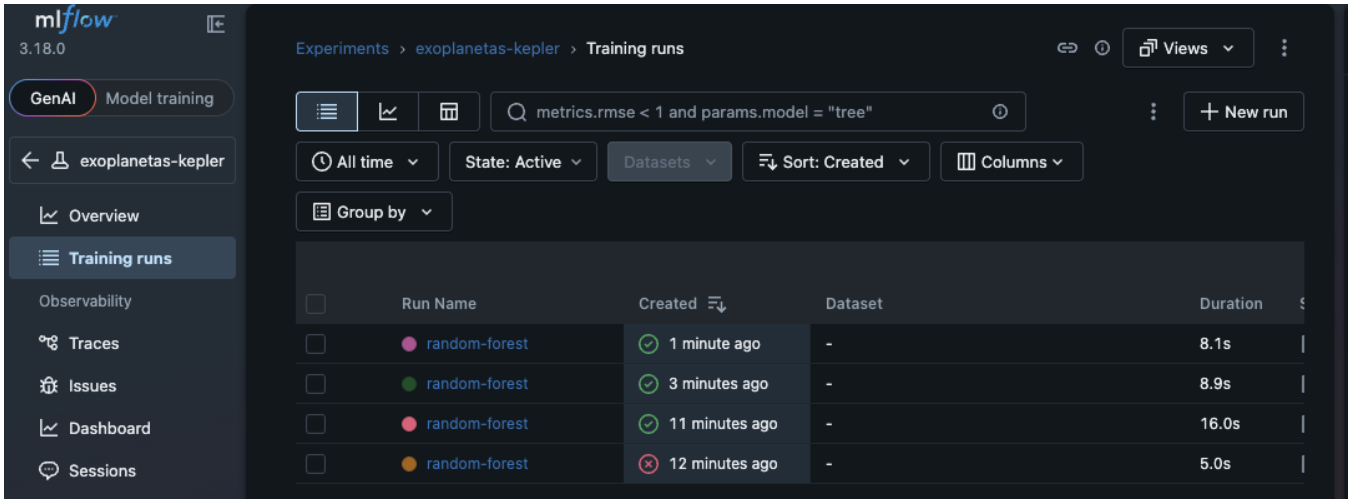
Evidencia 8. Imágenes publicadas automáticamente en ECR

Además de la imagen v1 subida a mano, aparece la imagen que publicó GitHub Actions con las etiquetas **latest** y el **SHA del commit** (f52ca3d…). Así cada versión de la imagen se puede rastrear hasta su commit.



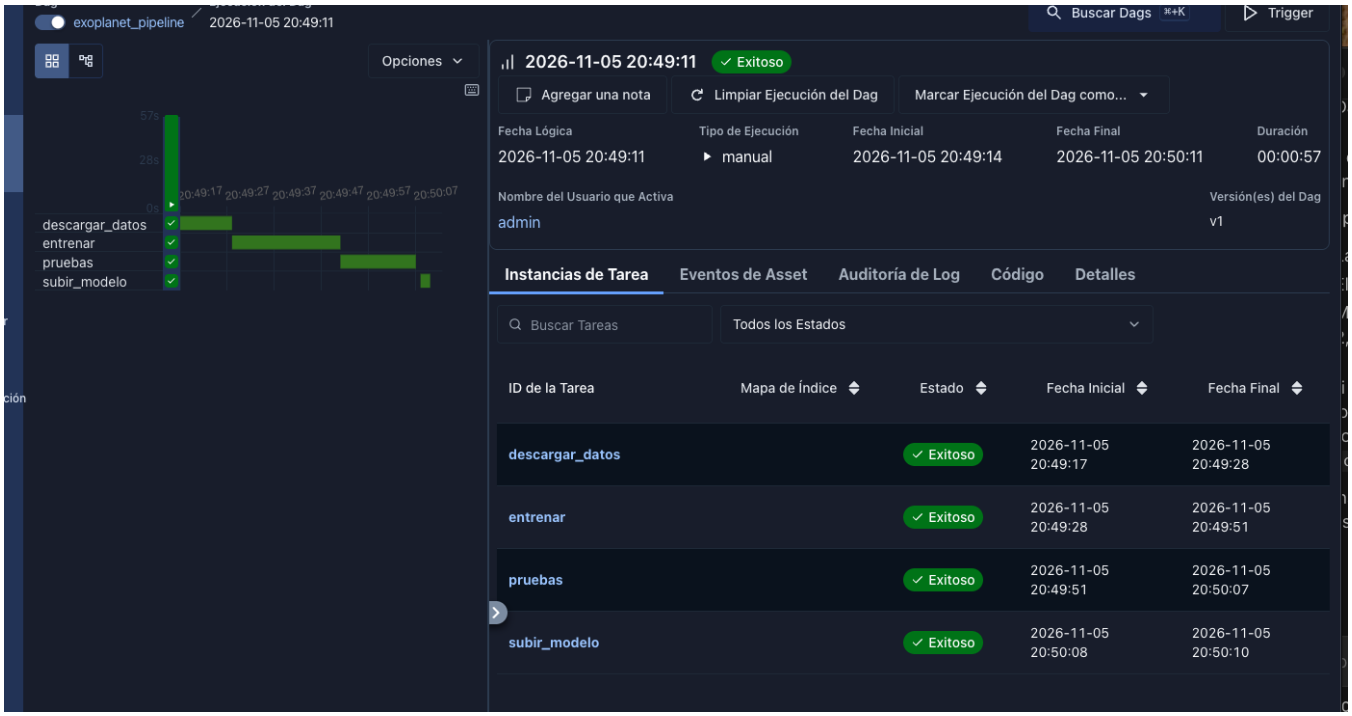
Evidencia 9. Entrenamientos registrados en MLflow (local)

El experimento *exoplanetas-kepler* guarda cada ejecución de *train.py* con sus parámetros y métricas (accuracy 0.913, F1 0.866, ROC-AUC 0.968). El modelo se registra como *exoplanet-champion* y sus artefactos se guardan en S3. El run marcado en rojo es el primer intento, que falló al serializar el modelo con skops y se corrigió declarando los tipos confiables.



Evidencia 10. Pipeline orquestado con Airflow (local)

El DAG *exoplanet_pipeline* (Airflow 3.1) encadena cuatro tareas: **descargar_datos** (*dvc pull* desde S3), **entrenar** (*train.py*, que registra el run en MLflow), **pruebas** (pytest) y **subir_modelo** (copia *champion.joblib* a S3). La ejecución manual terminó como **Exitoso** en 57 segundos, con las cuatro tareas en verde.



MLflow y Airflow desplegados en la nube (EC2)

En las evidencias 9 y 10 MLflow y Airflow corrían en la Mac, con datos y artefactos en S3. En esta etapa se movieron a un servidor EC2 para que todo el flujo viva en AWS.

Paso	Qué se hizo
Rol IAM	*mlops-ec2-role* con permisos solo para el bucket del proyecto (ListBucket, GetObject, PutObject, DeleteObject). El servidor no guarda access keys.
Red	Grupo de seguridad *mlops-server-sg*: solo el puerto 22 (SSH) abierto y únicamente para la IP de la autora.
Servidor	Instancia *m7i-flex.large* (2 vCPU, 8 GB) con Ubuntu 24.04 y 20 GB de disco gp3, elegible en el Free Plan.
Instalación	Script *deploy/setup_ec2.sh* del repositorio: crea los entornos de Python, instala MLflow, DVC y Airflow 3.1, y los registra como servicios de systemd para que arranquen solos.
Acceso	MLflow (puerto 5001) y Airflow (puerto 8080) no se exponen a internet: se abren desde la Mac con un túnel SSH (*ssh -L*).

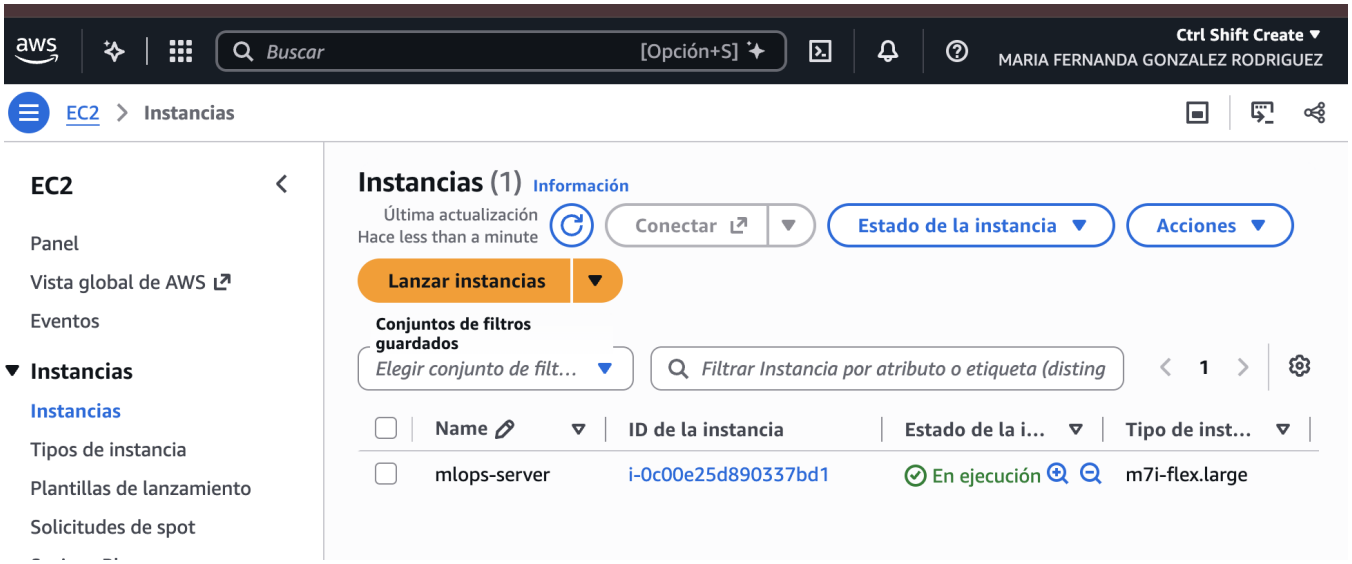
Evidencia 11. Verificación de recursos antes de desplegar

Antes de crear el servidor se revisó con la CLI que en us-east-2 no quedara nada cobrando: sin servicios ni tareas de ECS, instancias EC2, load balancers, clústeres EKS, NAT gateways ni IPs elásticas.



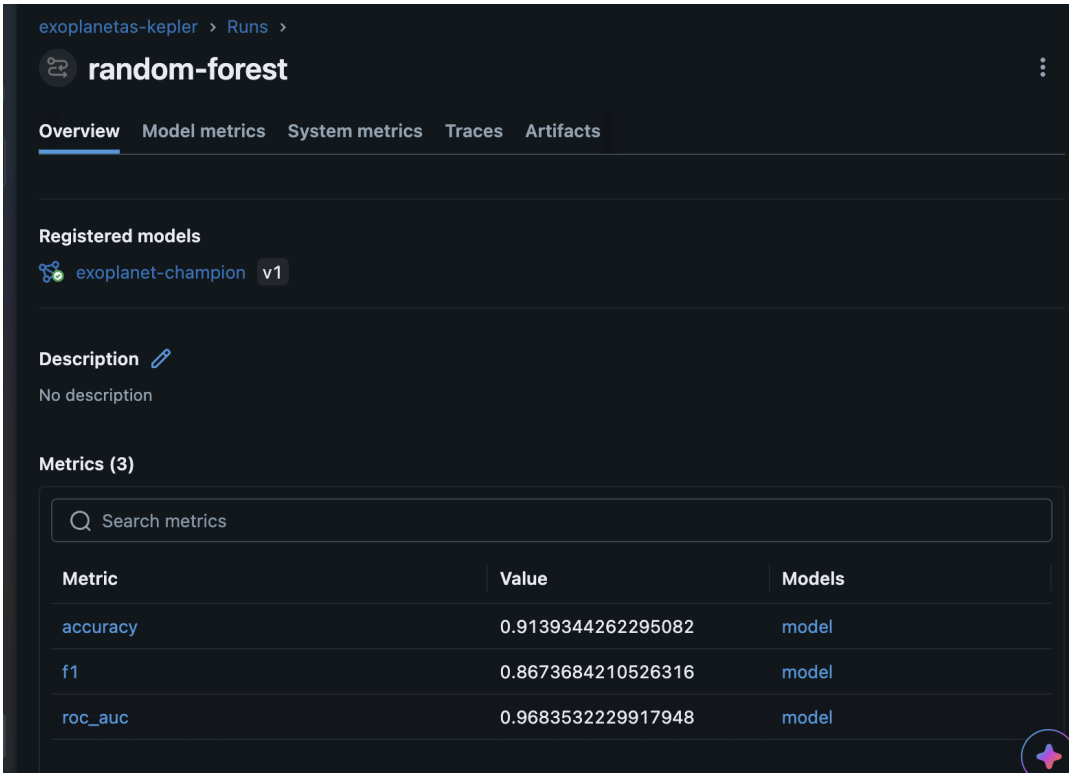
Evidencia 12. Servidor EC2 para MLflow y Airflow

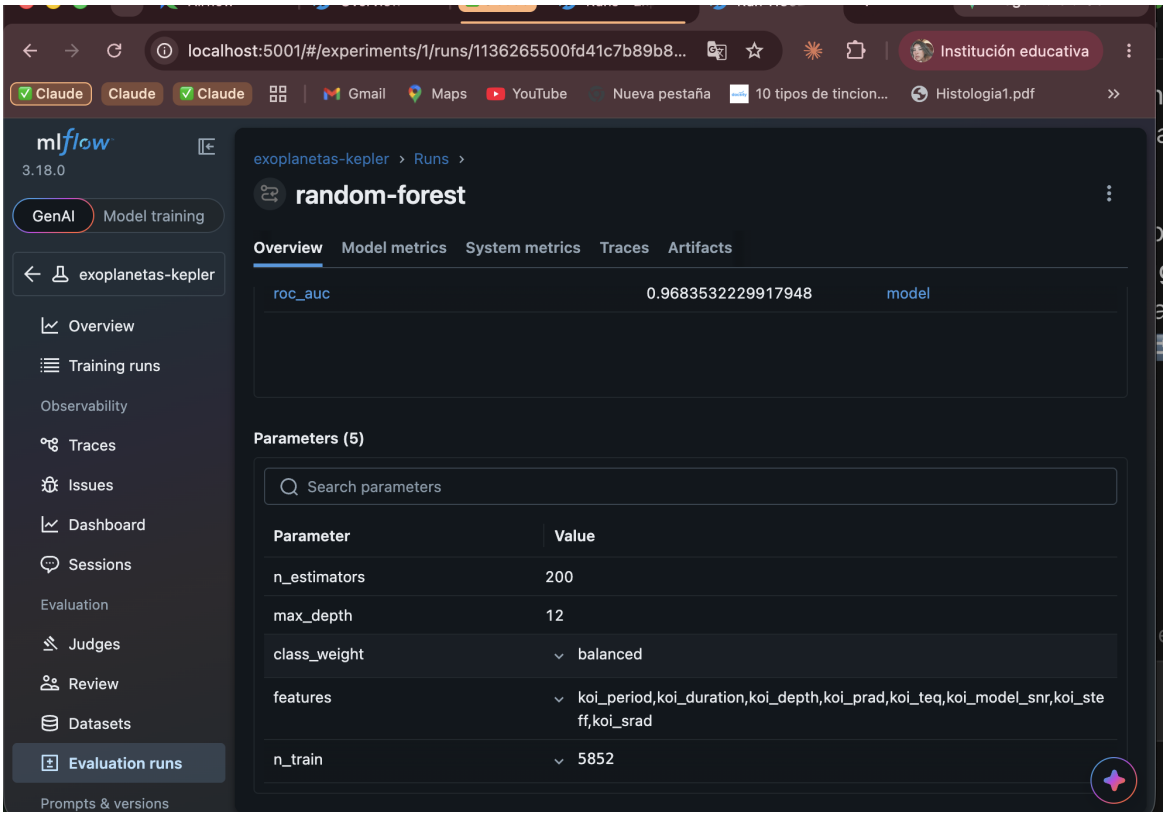
La instancia *mlops-server* (*m7i-flex.large*) en ejecución en us-east-2. Ahí corren MLflow y Airflow como servicios del sistema.



Evidencia 13. Entrenamiento registrado en MLflow desde EC2

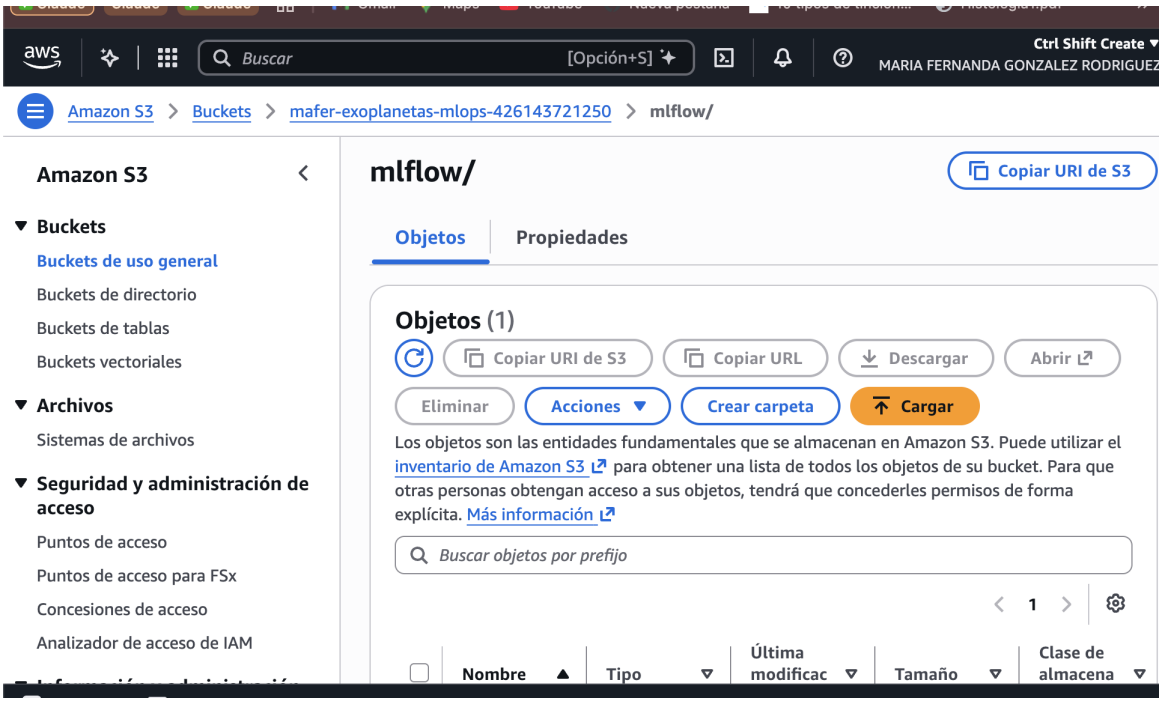
El DAG *exoplanet_pipeline*, ejecutado en el Airflow del servidor, corrió *train.py* y registró el run *random-forest* en el MLflow de EC2: accuracy 0.914, F1 0.867 y ROC-AUC 0.968, con el modelo registrado como *exoplanet-champion* v1. La segunda imagen muestra los parámetros: 200 árboles, profundidad máxima 12, pesos balanceados y 5,852 registros de entrenamiento.





Evidencia 14. Artefactos de MLflow en S3

El servidor de MLflow guarda los artefactos del modelo en la carpeta *mlflow/* del bucket, usando el rol IAM de la instancia.



Análisis

¿Por qué servicios individuales y no SageMaker? La cuenta está en el Free Plan, donde SageMaker está restringido, mientras que S3, ECR, ECS y CloudWatch sí están disponibles. Además, este enfoque reutiliza directamente lo construido antes: la misma imagen Docker, el mismo workflow de GitHub Actions y los mismos YAML de Kubernetes, que podrían desplegarse en EKS cambiando solo la imagen por la de ECR.

Ventajas: control total de cada pieza y costo muy bajo (Fargate con 0.25 vCPU y 1 GB, sin load balancer, cuesta centavos por hora). **Desventaja:** hay que configurar e integrar cada servicio a mano (roles IAM, red, grupos de seguridad, logs), algo que SageMaker resuelve con infraestructura administrada.

Costos: al terminar las capturas se eliminó el servicio de ECS y se apagó la instancia EC2 (*stop*), que son lo único que cobra por hora. El disco del servidor apagado, el bucket, la imagen de ECR y los logs cuestan centavos al mes y se descuentan de los créditos del Free Plan.

Seguridad: el servidor usa un rol IAM con permisos mínimos sobre un solo bucket, no guarda access keys y solo acepta SSH desde una IP. MLflow no tiene autenticación, por eso se accede a él con un túnel SSH en lugar de abrir sus puertos.

Problemas resueltos: (1) zsh interpretó *$ACCOUNT:r* como un modificador y borró el texto *:r* del ARN del rol; se corrigió el JSON de la tarea. (2) *dvc push* requería el plugin *dvc-s3* dentro del entorno virtual. (3) MLflow 3 guarda los modelos de sklearn con skops y exige declarar como confiables los tipos internos del Random Forest. (4) El Free Plan no permite *t3.medium*; se eligió *m7i-flex.large*, que sí es elegible. (5) El DAG tenía la ruta de la Mac escrita a mano; se hizo portable calculando la carpeta del proyecto a partir del propio archivo. (6) La sincronización de iCloud bloqueaba archivos de Git y regresaba versiones viejas; se resolvió quitando el *index.lock* y volviendo a escribir el archivo.